

Image & Sequential Data

2026 Winter DL Session 5주차 Part 1



고려대학교
KOREA UNIVERSITY

KUBIG
DATA SCIENCE & AI

Copyright © 2026
by KUBIG

목차

KUBIG 2026 Winter DL Session

- 01 들어가기 전에,,
- 02 CNN
- 03 Deeper Layers
- 04 Sequential Data
- 05 RNN & LSTM
- 06 다음주차 예고

01 들어가기 전에,,

01. 들어가기 전에,,

4주차 과제 발표

22기 박준영 님

23기 정윤서 님

모두 과제하느라 수고많으셨습니다 ! :)

01. 들어가기 전에,,

4주차 과제의 의도

1. 네트워크 정의 (TwoLayerNet → MultiLayerNet: 가중치 초기화 및 감소 weight_decay)
2. Layers(Affine, Relu, Softmax) + Sigmoid
3. Optimizers (SGD, Momentum, AdaGrad) + Adam
4. Regularization (BatchNormalization, Dropout)
5. MNIST 데이터에 정규화 기법 적용별 효과 비교 (정규화 기법 적용 전 vs 후) (정규화 기법별 비교)

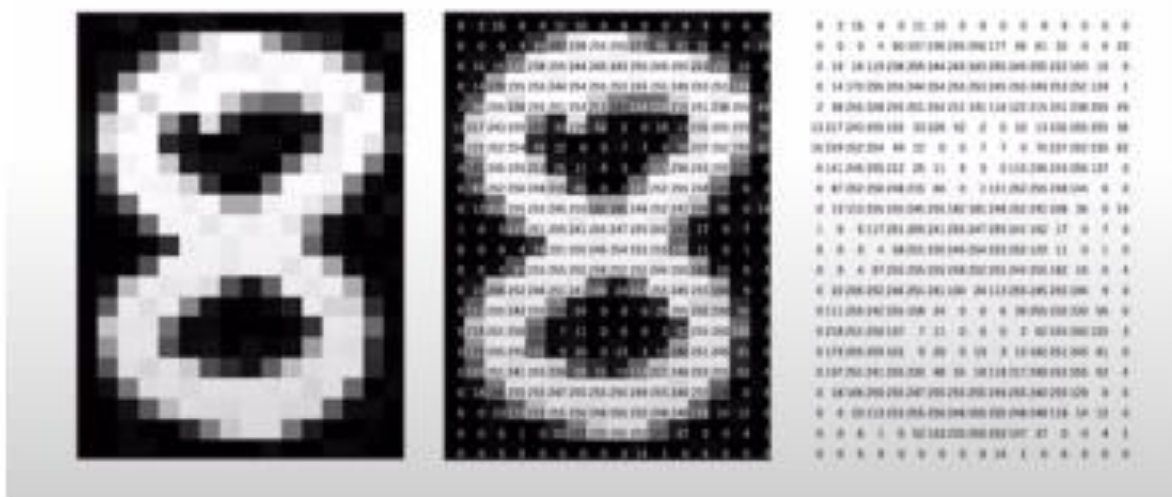
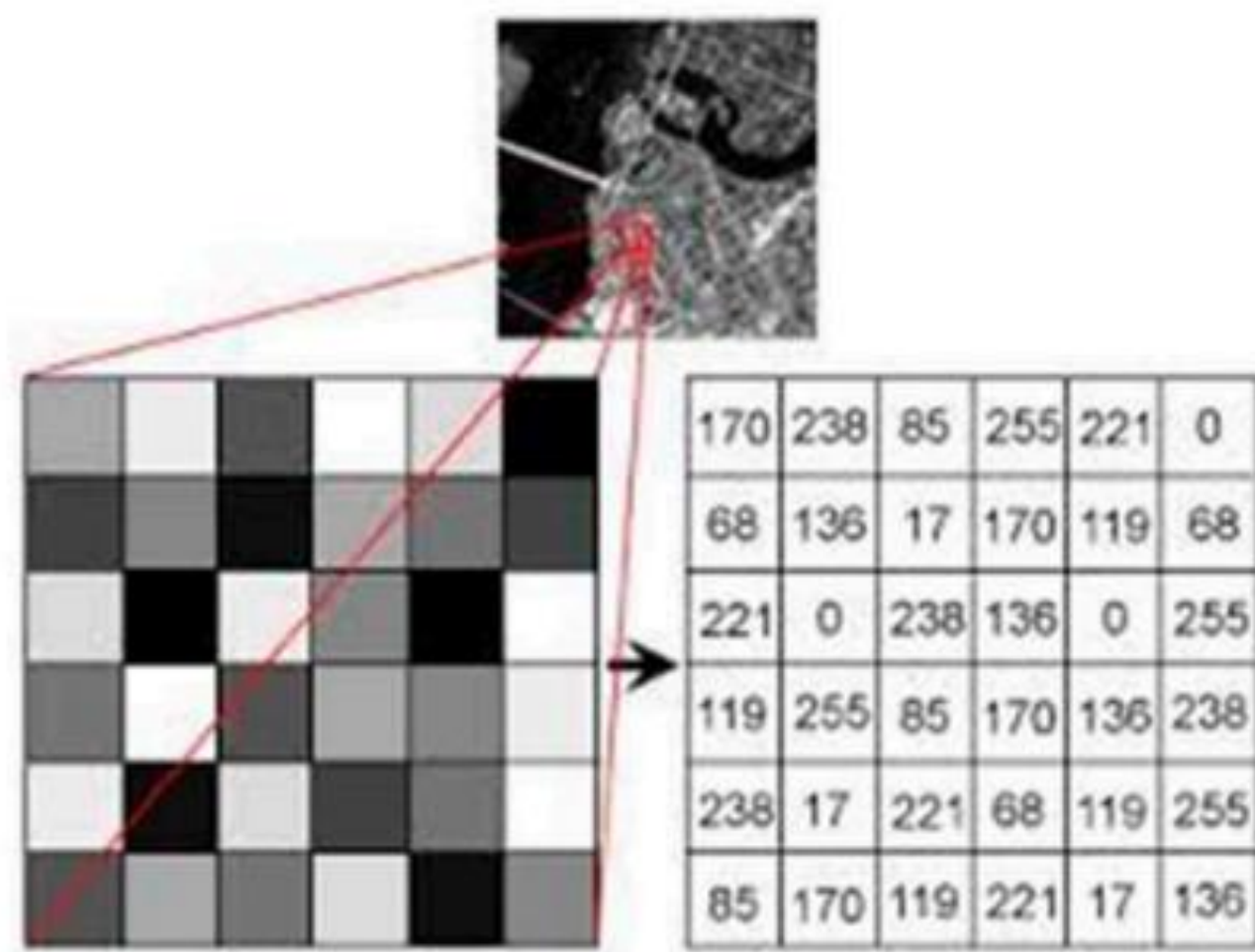
5주차 학습 목표

1. 이미지 처리에 강점을 가지는 CNN이 무엇인지 알아본다
2. RNN과 LSTM과 같은 Autoregressive model이 Sequential data를 처리하는 과정을 이해한다.

02 CNN

02. CNN - Image Data Review

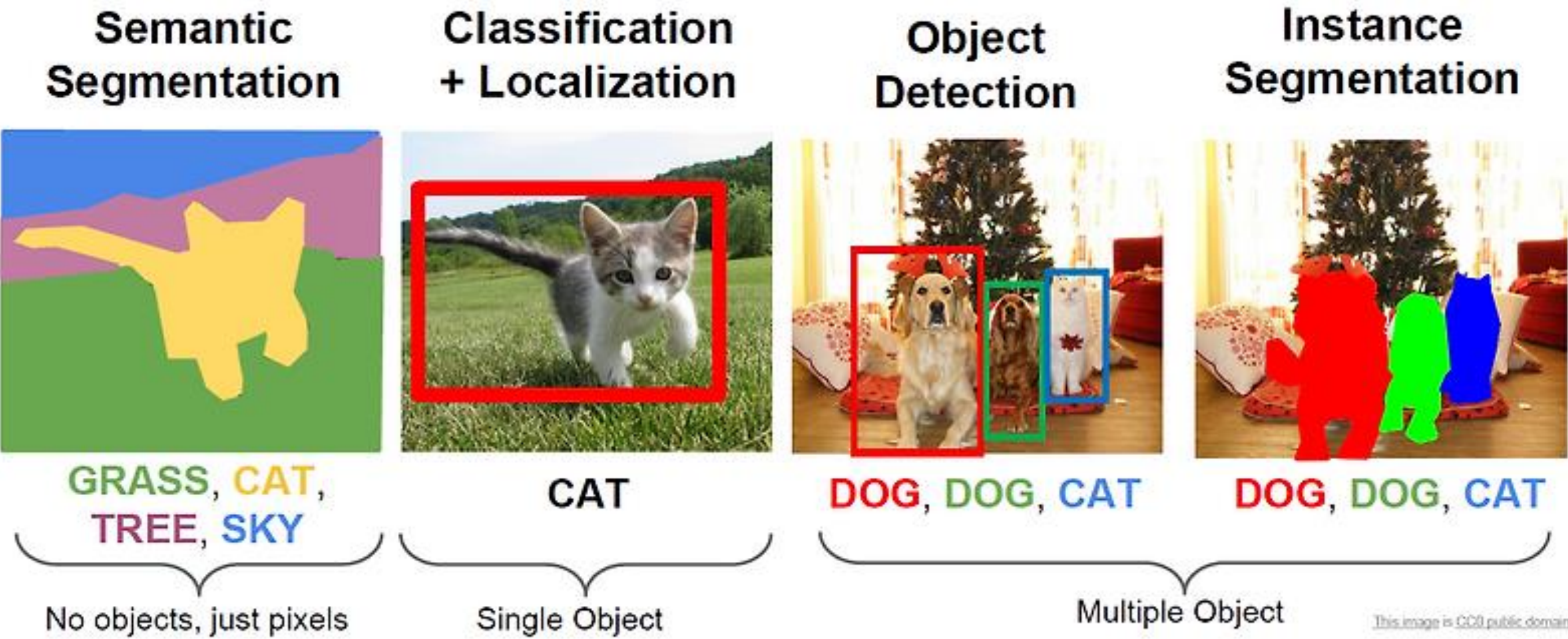
Image data



- 픽셀로 따지면 가로 W개, 높이가 H개
- $\#(\text{pixel}) = H * W$
 - 픽셀 수 $\uparrow$, 화질(resolution) $\uparrow$
 - 픽셀 값 = 밝기 값
 - 0~255 사이, 0: 검정 255: 하양

02. CNN - Image Data Review

Computer Vision Tasks

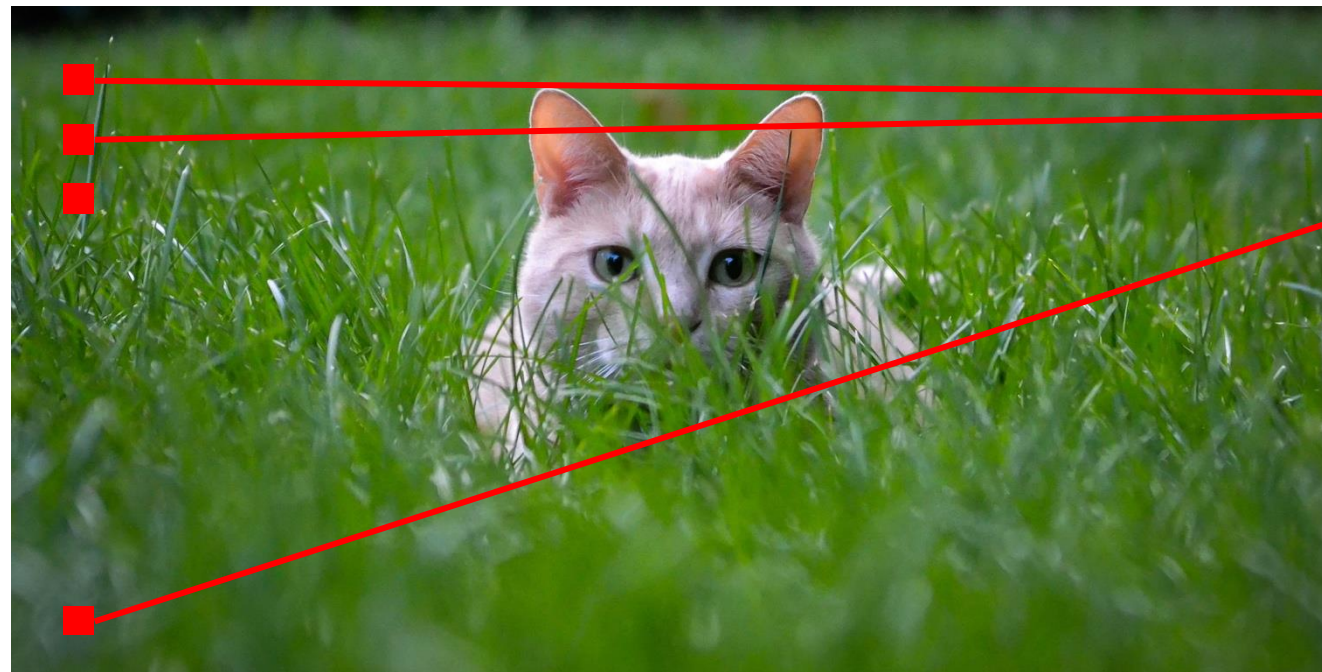


02. CNN - Image Data Review

MLP on CV?

- 메모리 비효율적
- Optimization 어려움

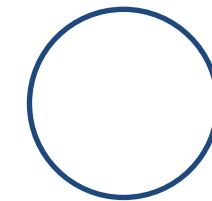
MLP에서는 모든 픽셀에 대해
각각 파라미터가 필요



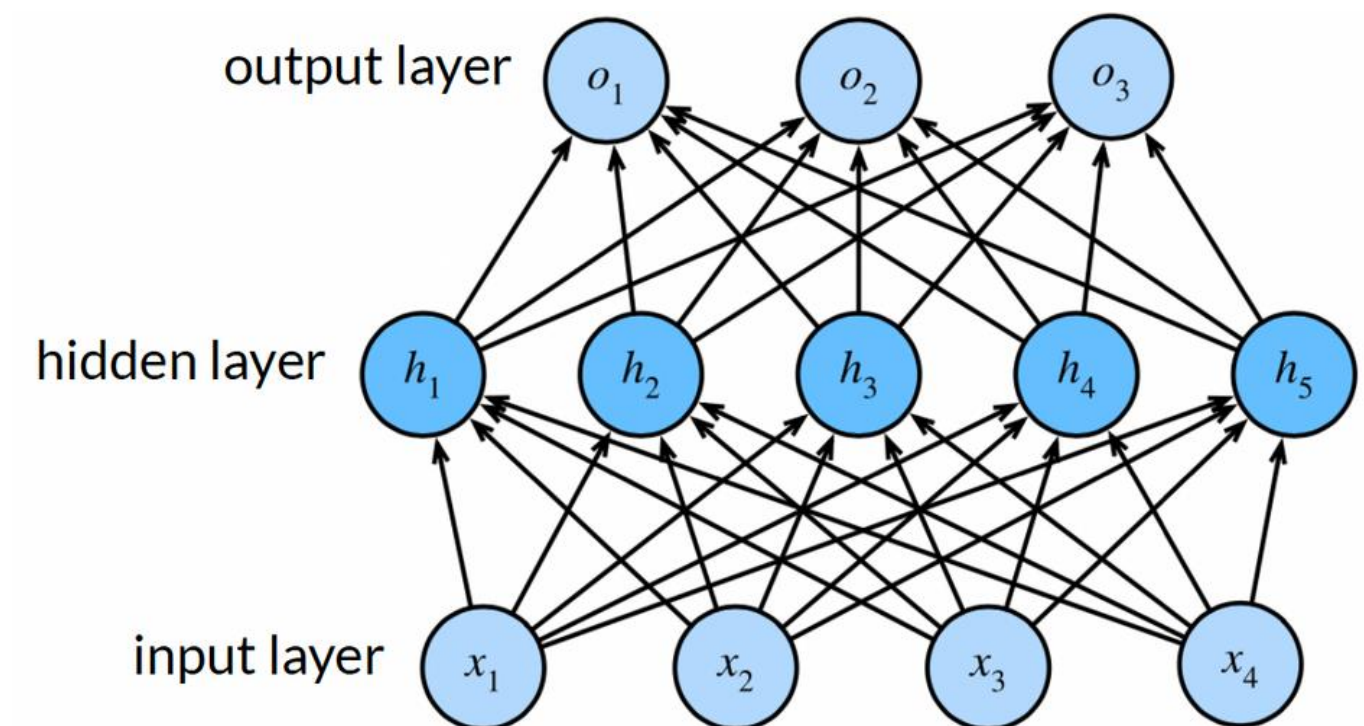
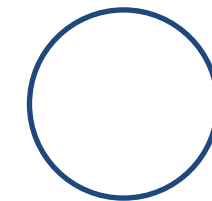
x_j

h_i

$$= \sum_j w_{ij} x_j$$

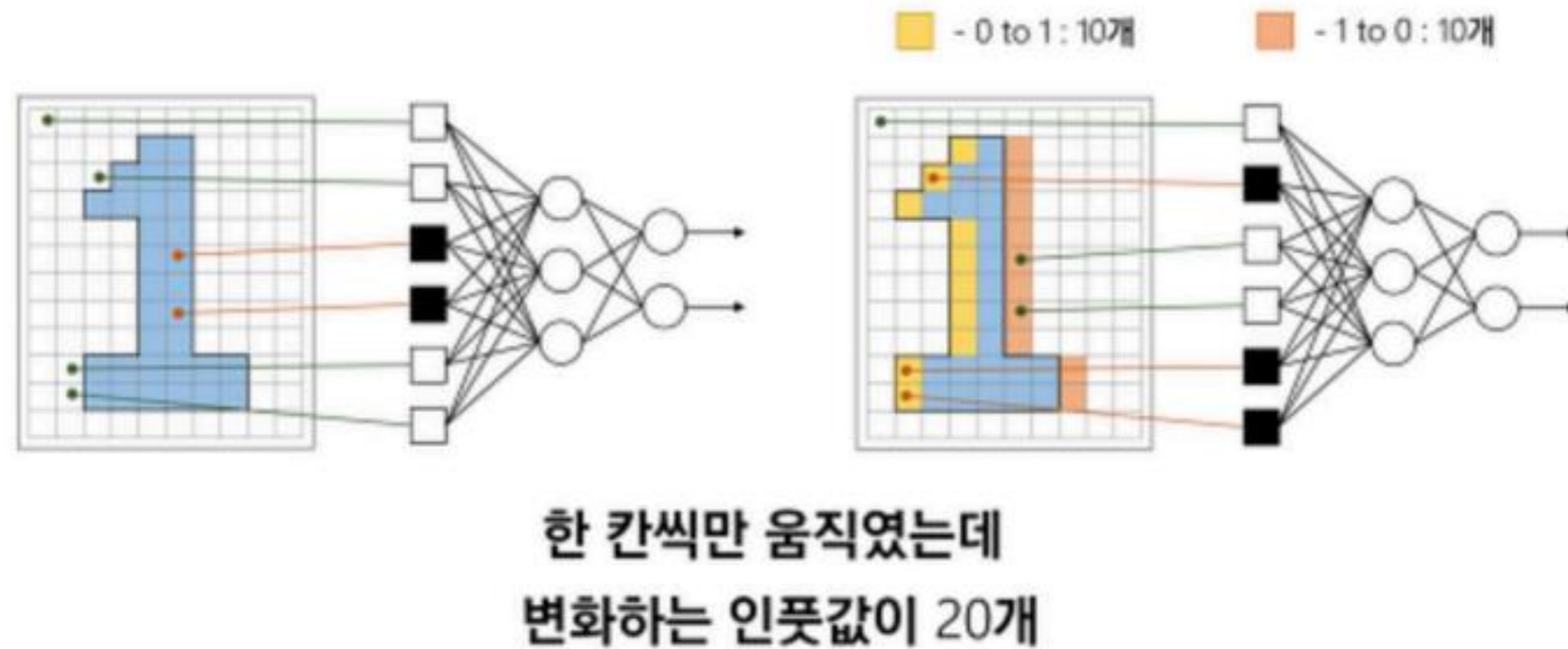


⋮



02. CNN - Image Data Review

MLP on CV?

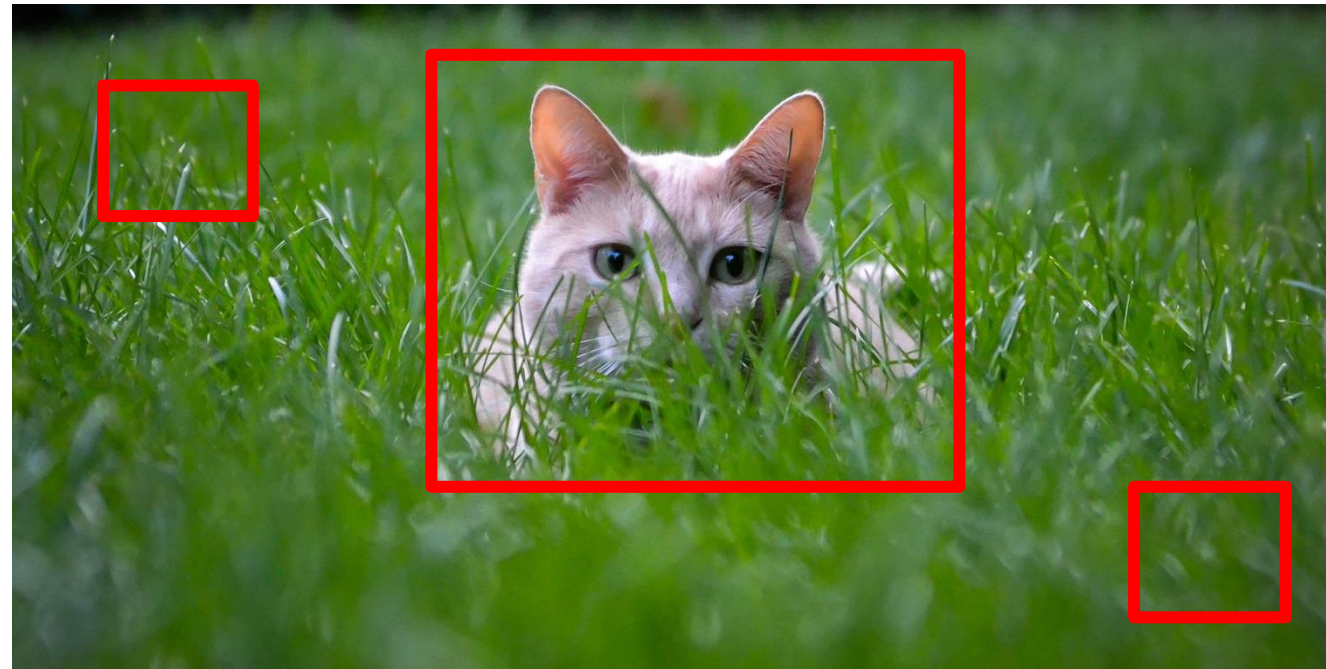


Why difficult?

이미지가 조금만 차이 나더라도 input의 변화가 매우 크게 발생함.

02. CNN - Image Data Review

Principles for Visual Domain



모든 픽셀에 대한 정보를 활용하는 모델 (MLP)는
허위 상관(Spurious Correlation) 문제에 취약

- 이미지 데이터: 먼 지역의 내용(특징)은 무관

02. CNN - Image Data Review

Principles for Visual Domain

→ Locality principle

먼 지역의 내용(특징)은 무관하므로,
이미지의 특정 부분을 인식할 때 그 부분과 멀리 떨어진 다른 지역의 정보는 중요하지 않다고 간주된다.

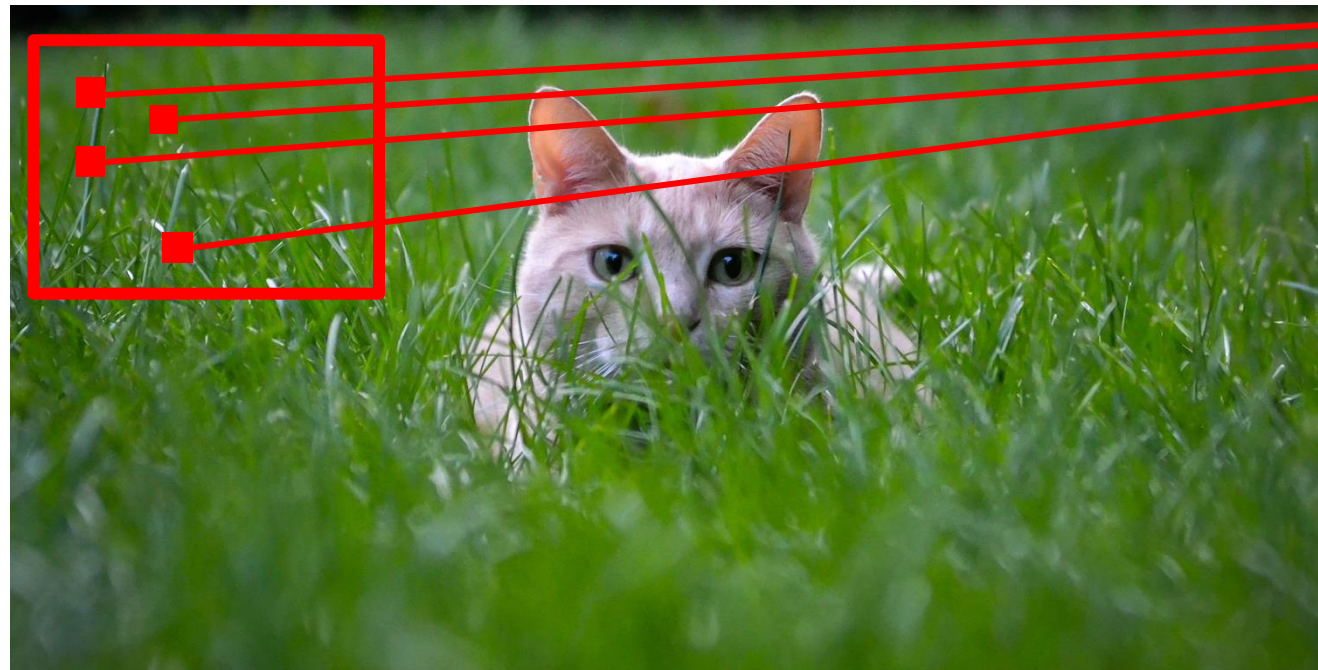
이미지를 스캔하면서 각 Image patch(조각)에 점수를 매긴다.
= 이미지의 구조는 국소적인 정보를 기반으로 처리될 수 있다.
= Local Info(특정 영역의 특징)을 활용한 효율적인 학습을 목표.

→ CNN(Convolutional Neural Network)은 Locality Principle에 기반하여 이미지를 처리!

02. CNN - Image Data Review

Principles for Visual Domain

→ Locality principle



h_1

h_2

h_3

이미지를 스위핑하여 각
패치(patch)에 점수 할당 가능

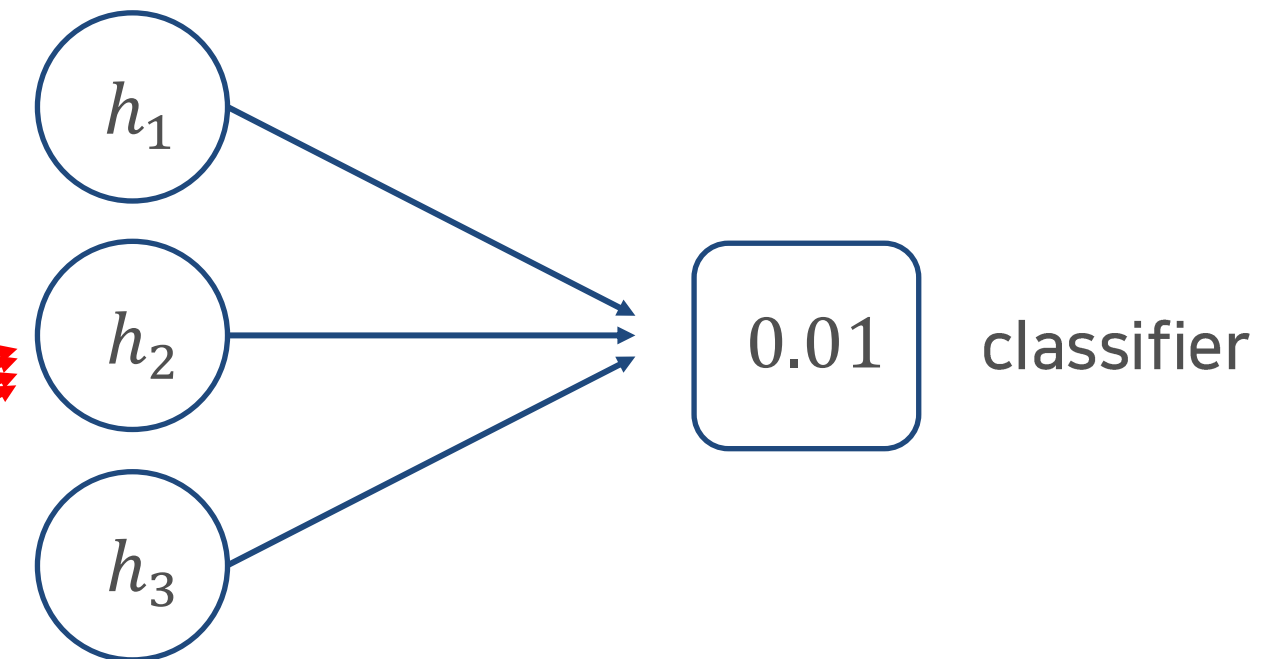
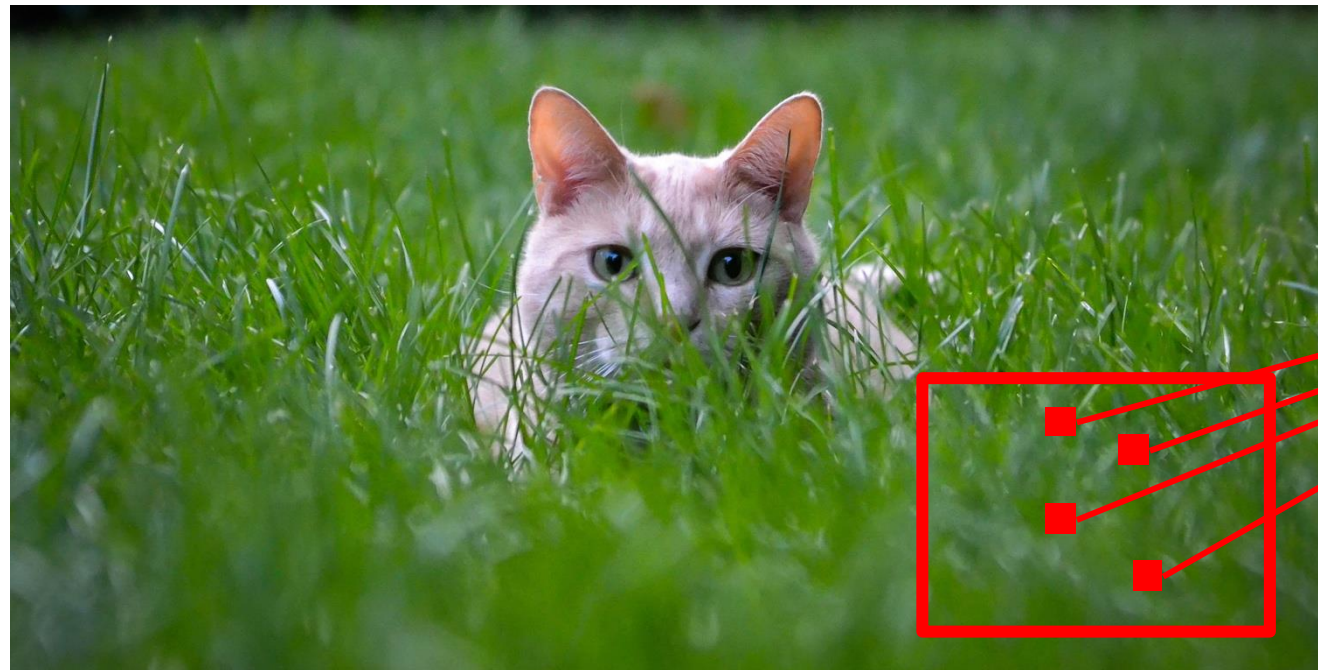
0.01

classifier

02. CNN - Image Data Review

Principles for Visual Domain

→ Locality principle

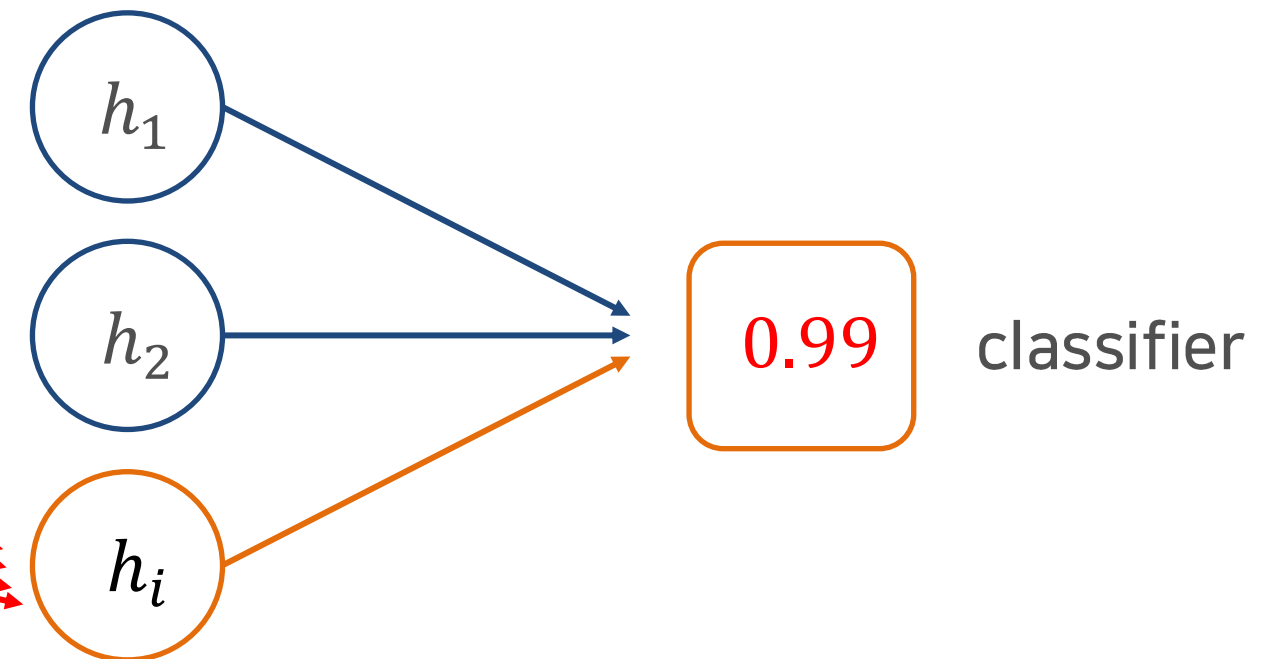
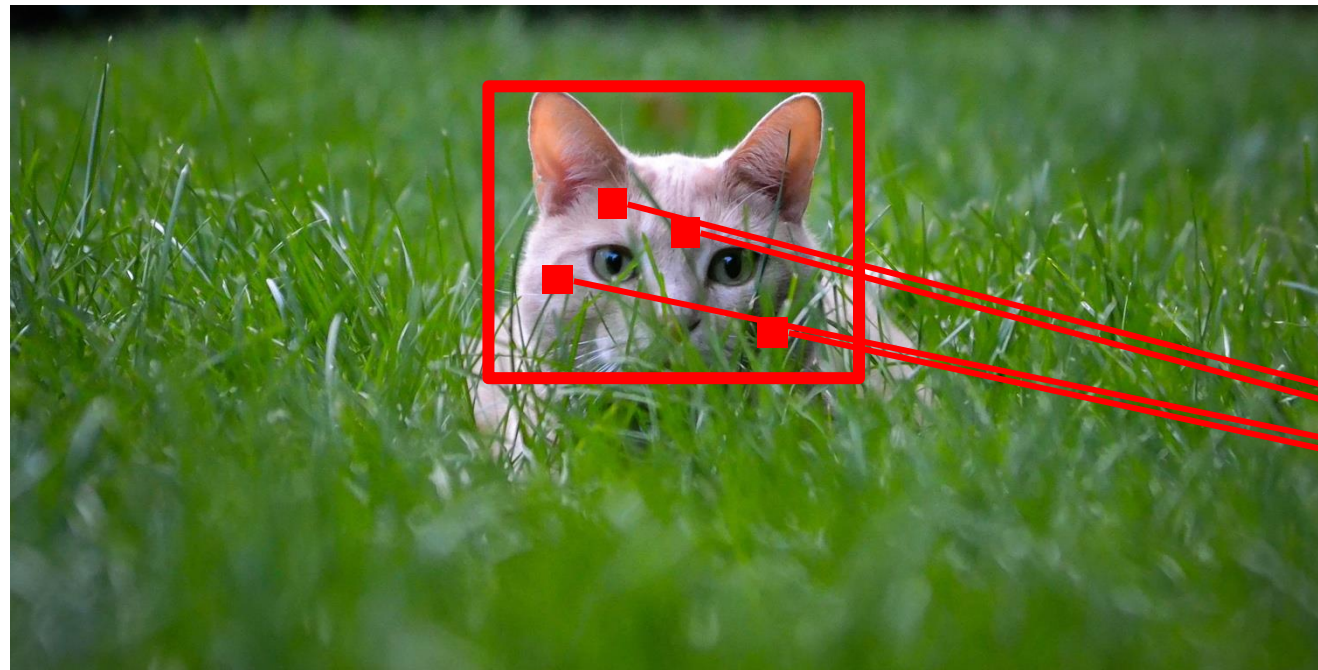


이미지를 스위핑하여 각
패치(patch)에 점수 할당 가능

02. CNN - Image Data Review

Principles for Visual Domain

→ Locality principle



02. CNN - Image Data Review

Principles for Visual Domain

→ Locality principle

먼 지역의 내용(특징)은 무관하므로,
이미지의 특정 부분을 인식할 때 그 부분과 멀리 떨어진 다른 지역의 정보는 중요하지 않다고 간주된다.

→ Spatial Invariance

이미지 내 객체의 위치에 의존하지 않아야 한다.

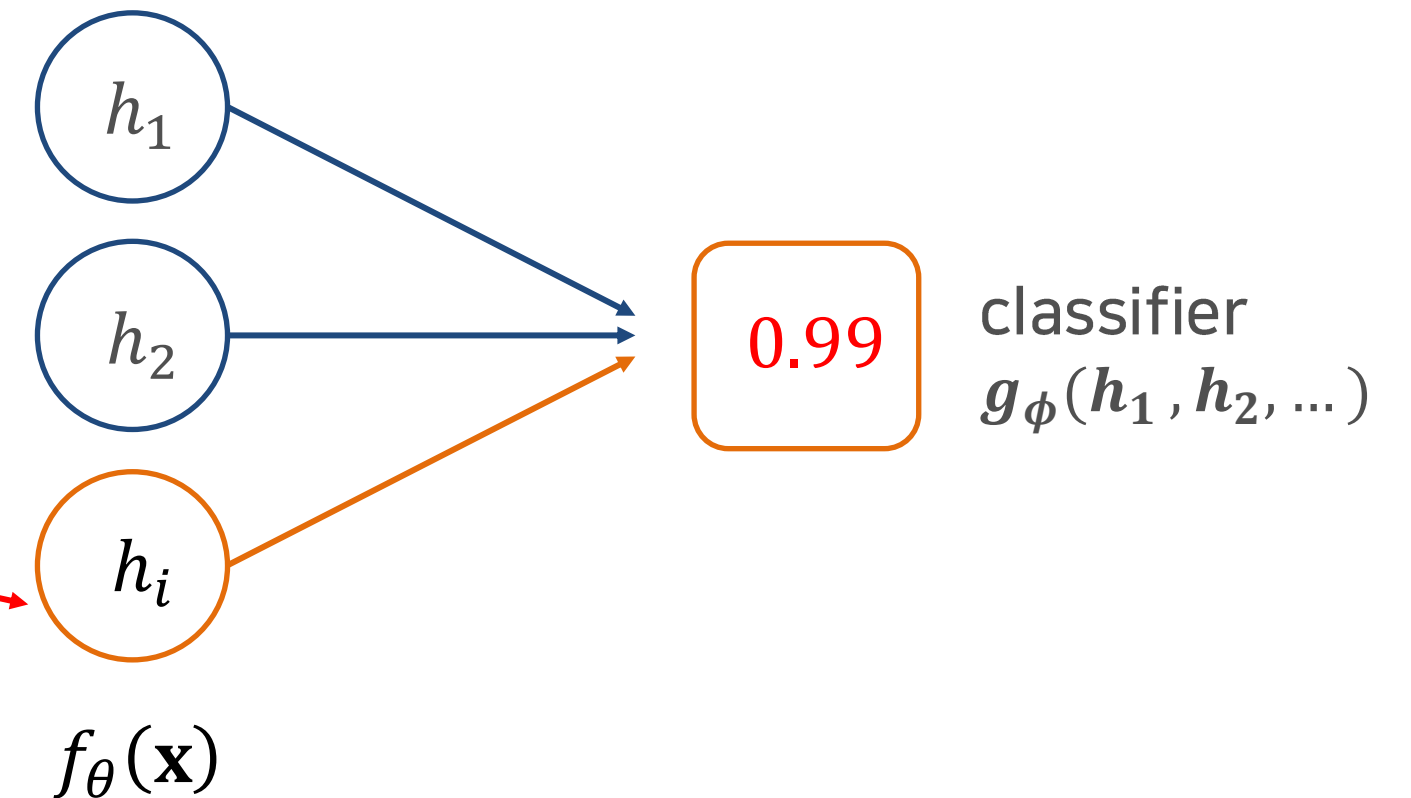
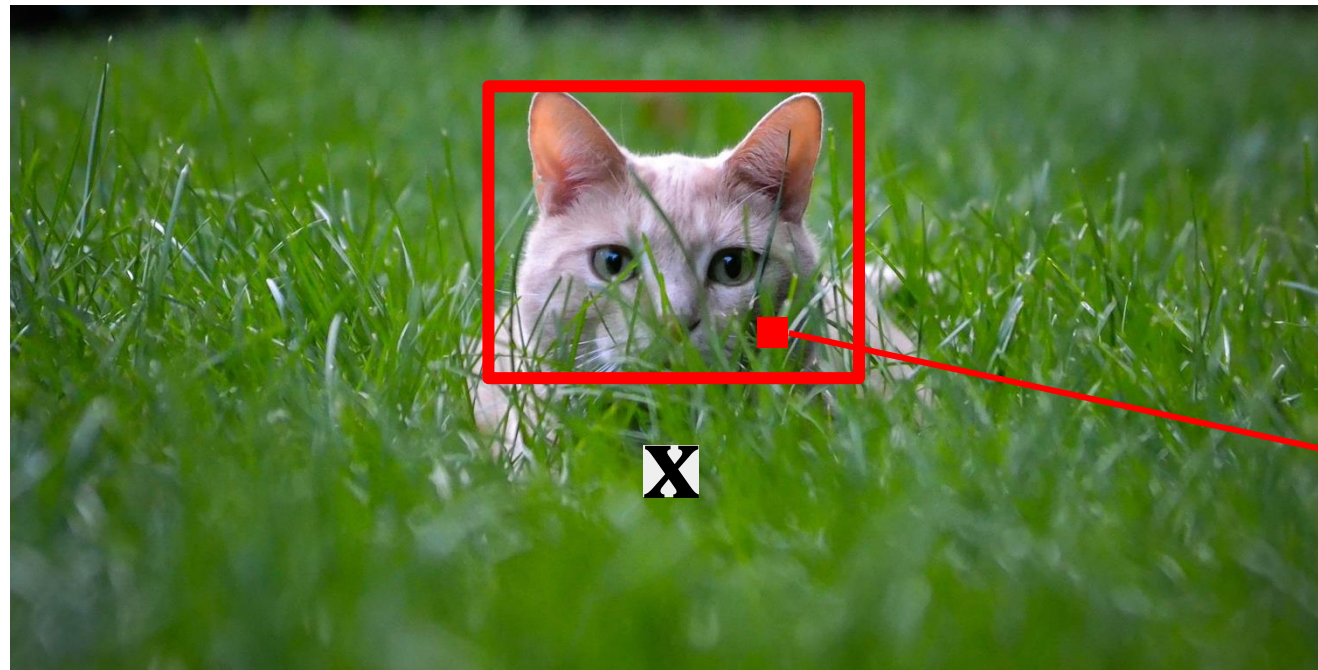
what object looks like does not depend upon *where the object is located*

Translation Equivariance & Translational Invariance

02. CNN - Image Data Review

Principles for Visual Domain

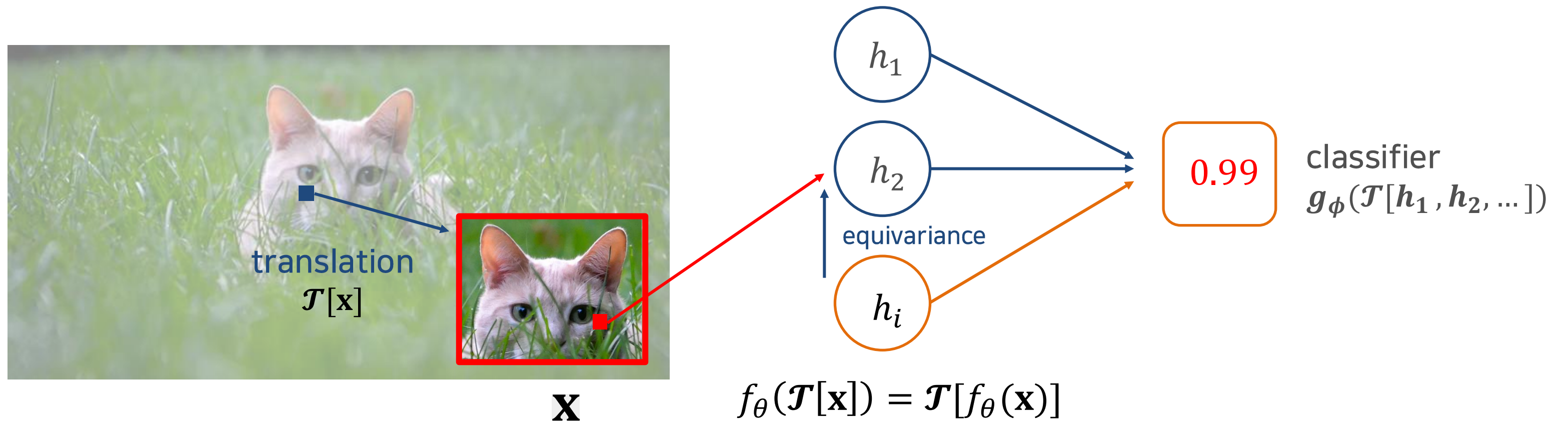
→ Locality principle



02. CNN - Image Data Review

Principles for Visual Domain

→ Translational Equivariance



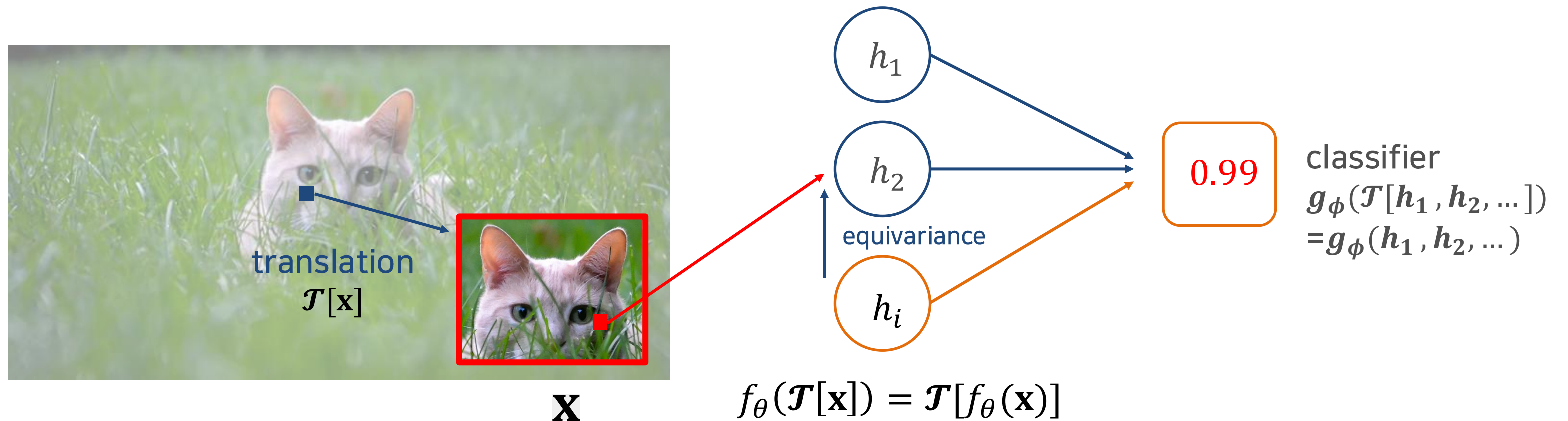
Translational Equivariance

: 객체가 이동(translate)하면, detector f_{θ} 는 **translate된 패치**에서 정보를 포착

02. CNN - Image Data Review

Principles for Visual Domain

→ Translational Equivariance + Invariance



Translational Invariance

: 그에 따라 분류기 g_ϕ 는 translate된 패치의 정보를 이용해서 **동일한 결과**를 출력!

02. CNN - Image Data Review

Principles for Visual Domain

**Semantic
Segmentation**



GRASS, CAT,
requires
equivariance

**Classification
+ Localization**



CAT
requires
invariance

**Object
Detection**



DOG, DOG, CAT
requires
equivariance

**Instance
Segmentation**



DOG, DOG, CAT
requires
equivariance

02. CNN - Image Data Review

MLP가 이미지 처리에 가지는 한계

1. 계산 효율성 (학습 parameter의 수)

= input data size의 수가 너무 많아서 학습 parameter가 너무 많아짐.

2. Locality 문제

pixel 하나는 정보값 X, pixel은 독립적으로 정보를 주는 게 아니라 주변의 pixel과 합쳐져서 정보를 주게 됨 = Spatial relation

→ Flatten해서 나열해버리면 위치정보가 사라지게 됨. (Serialization의 2차원 벡터로 표현)

→+여기에다가 weight sum을 하면, 위치정보가 또 사라지게 됨.

3. Translation 문제

Translation = 일부 변형을 주더라도 같은 그림으로 인식하는 것 (예시: 한쪽에 치우쳐진 사진 -> 같은 그림으로 인식 X)

if) 10000장의 데이터에 대해 학습을 시킨다면

→ 각각의 case에 대해 일일이 학습을 시켜야 함. (한쪽에 치우쳐진 거, 사이즈 바꾼 거, noise 있는 거, 회전된 거, 등등)

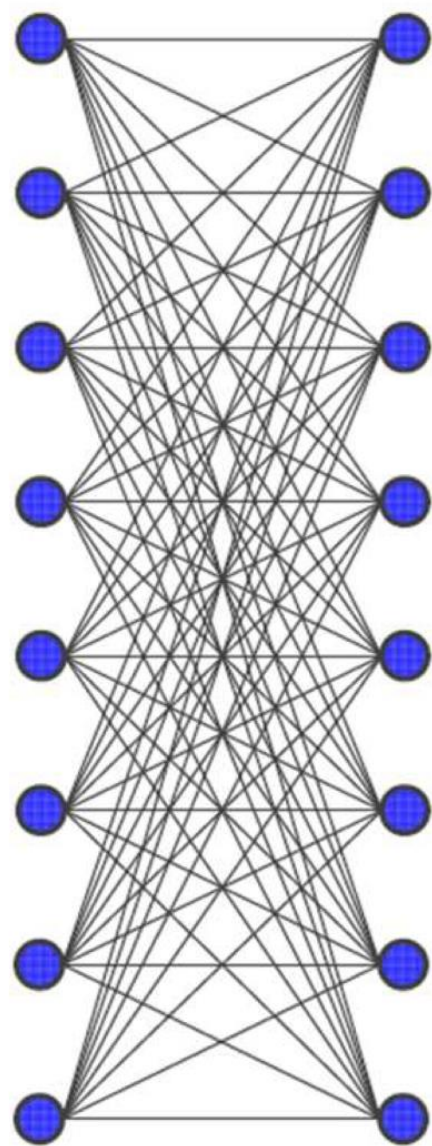
= 각 augmentation에 대해 각 데이터를 모두 학습을 시켜야 함.

→ 학습이 너무 어렵기 때문에, Translation을 할 수 있는 모델을 써야 함.

02. CNN - Image Data Review

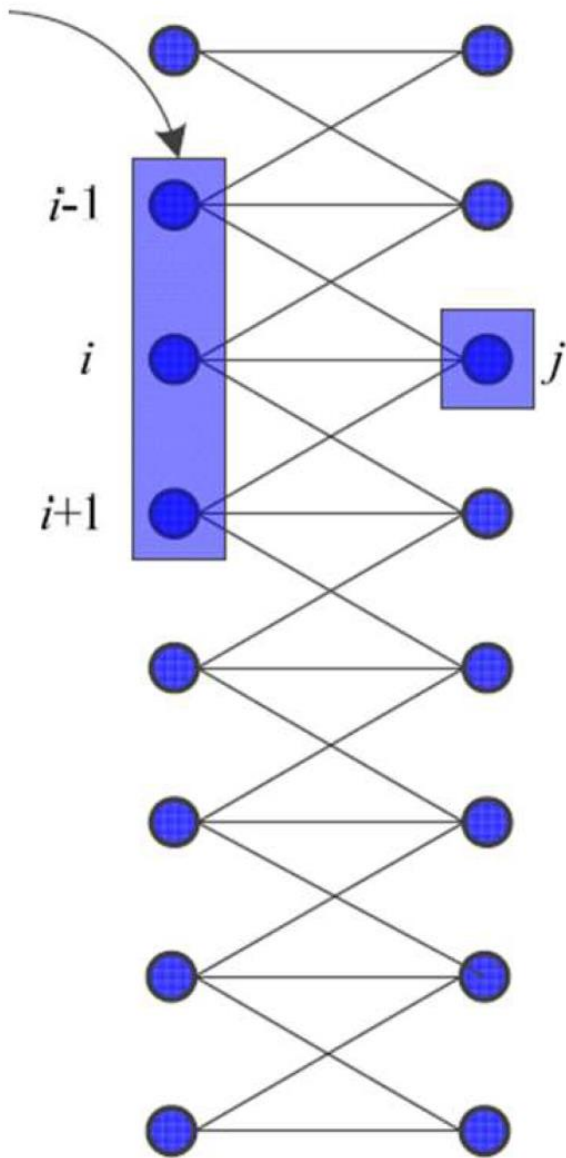
MLP가 이미지 처리에 가지는 한계

Layer $l-1$ Layer l

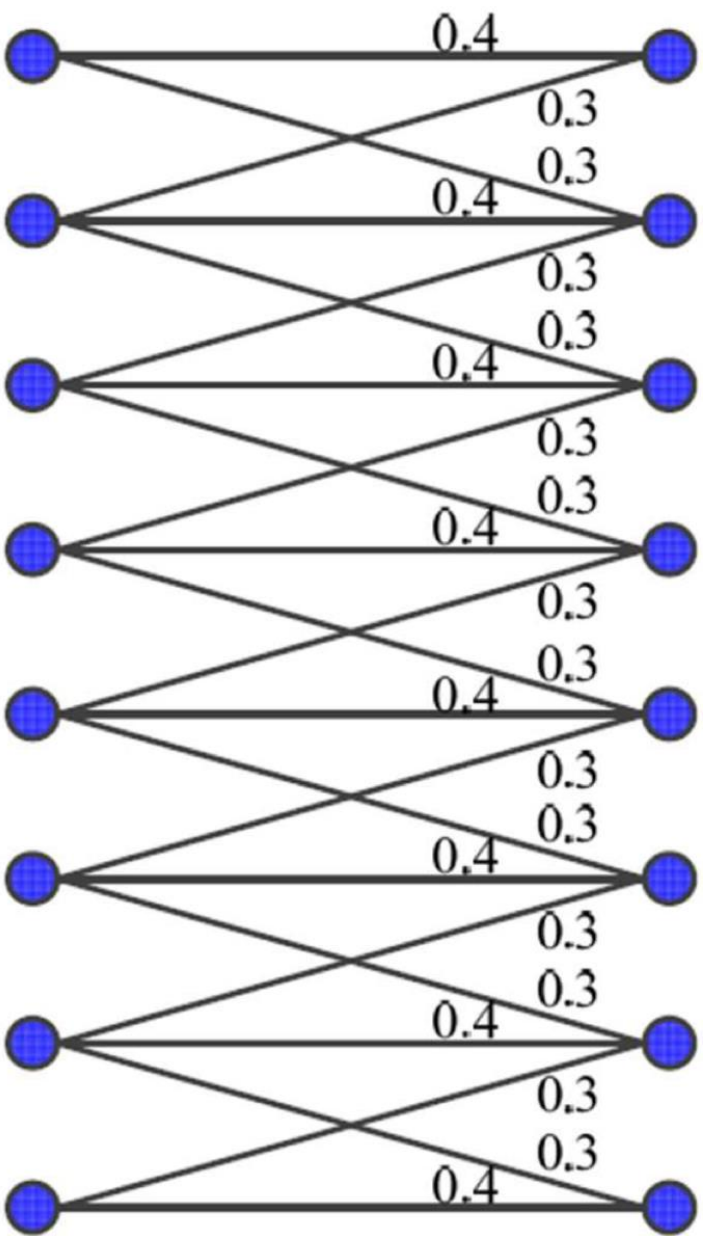


(a) DMLP (Fully-Connected)

Receptive Field Layer $l-1$ Layer l



(b) CNN (Partially-Connected)

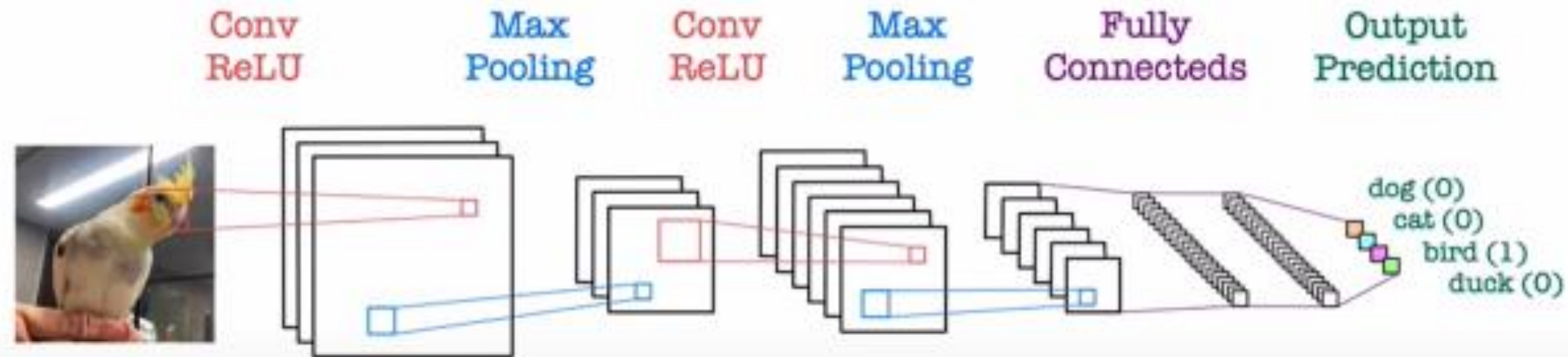


+ weight sharing!

02. CNN – Convolution Layer

CNN의 전체 architecture

- LeCun creates the first CNN **LeNet** for character recognitions at 1998.
- **AlexNet** which is ILSVRC 2012 winner has similar structure as LeNet.



- * Convolution (shared weights, dimension reduction)
- * Nonlinearity (ReLU)
- * Max pooling (translation invariance, dimension reduction for FC layer)
- * Fully connected layer (classification)

02. CNN – Convolution Layer

CNN이 이미지를 인식하는 단계



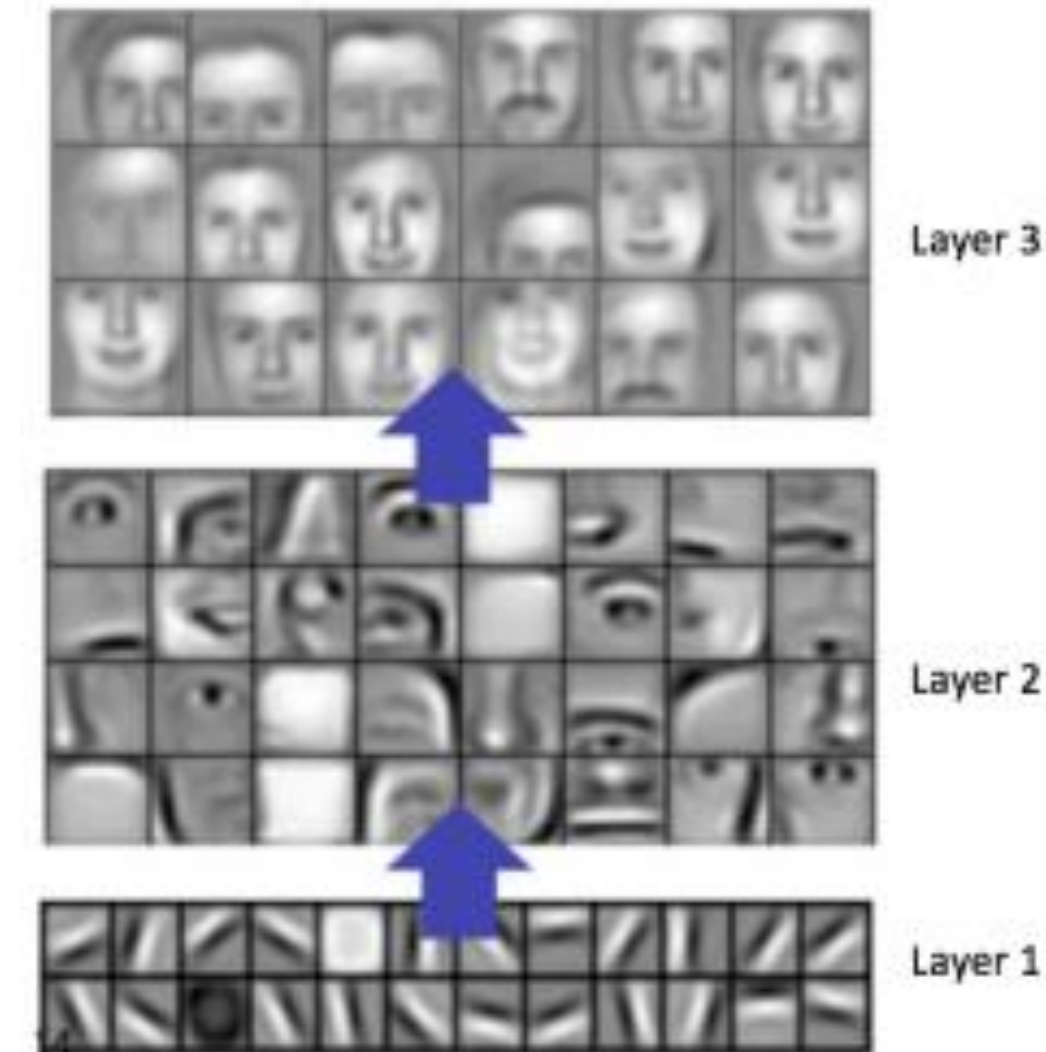
1단계 : 가로, 동그라미, 세모, 부드러움



2단계 : 눈, 코, 귀, 발



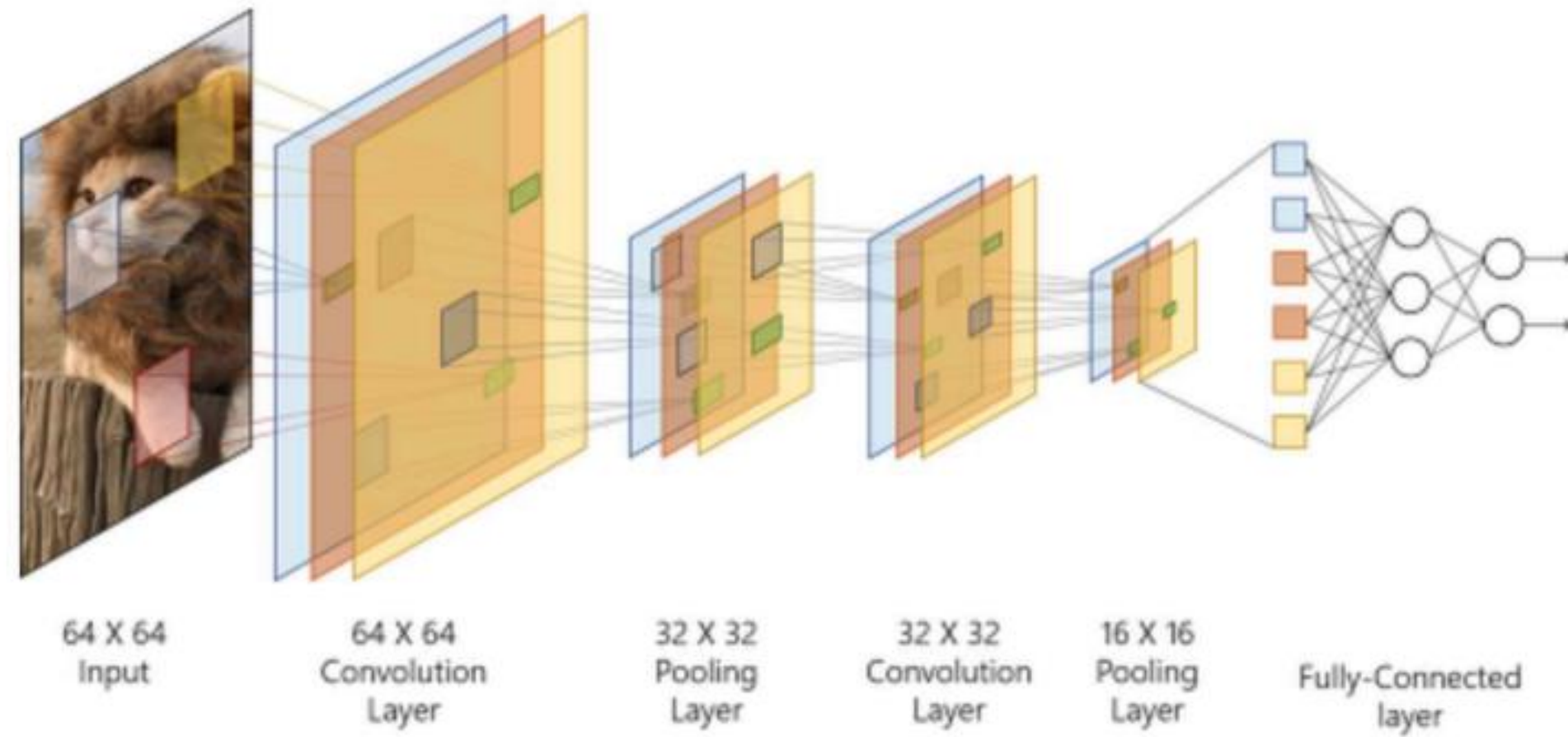
3단계 : 고양이!



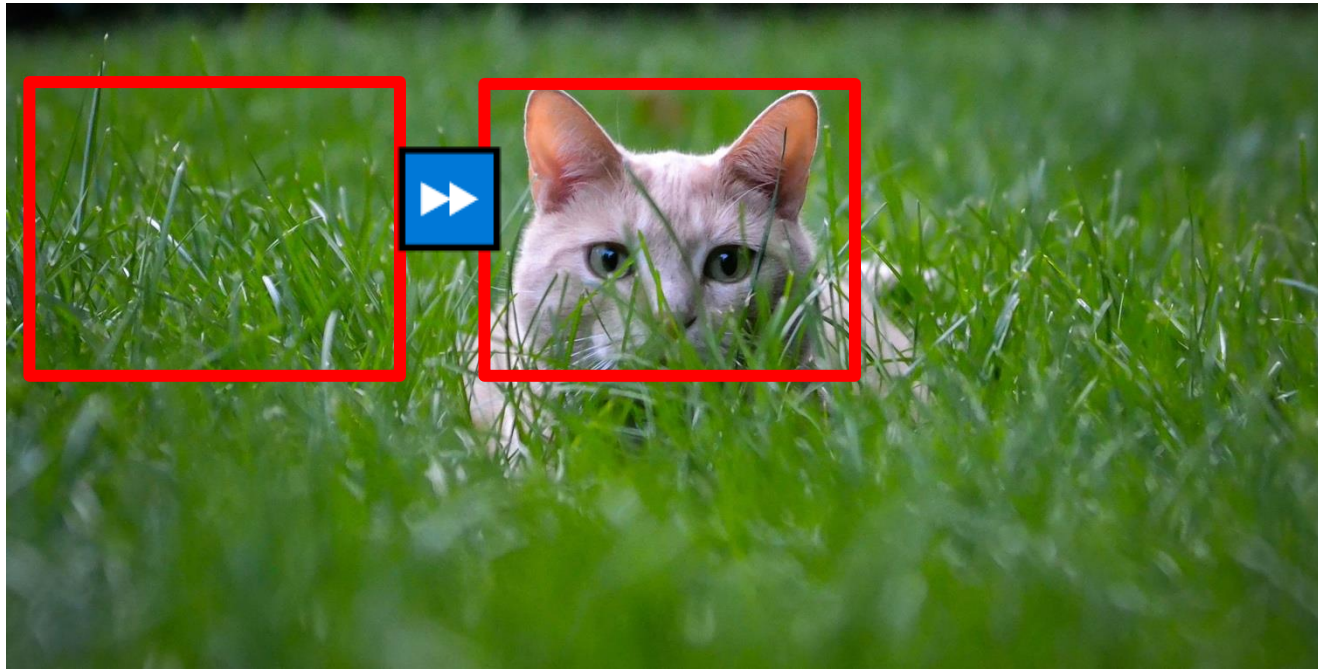
Conv Layer가 쌓일수록 '선' → '눈' → '고양이/얼굴' 처럼
점점 추상적인 특징을 조합

02. CNN – Convolution Layer

CNN이 이미지를 인식하는 단계



02. CNN – Convolution Layer

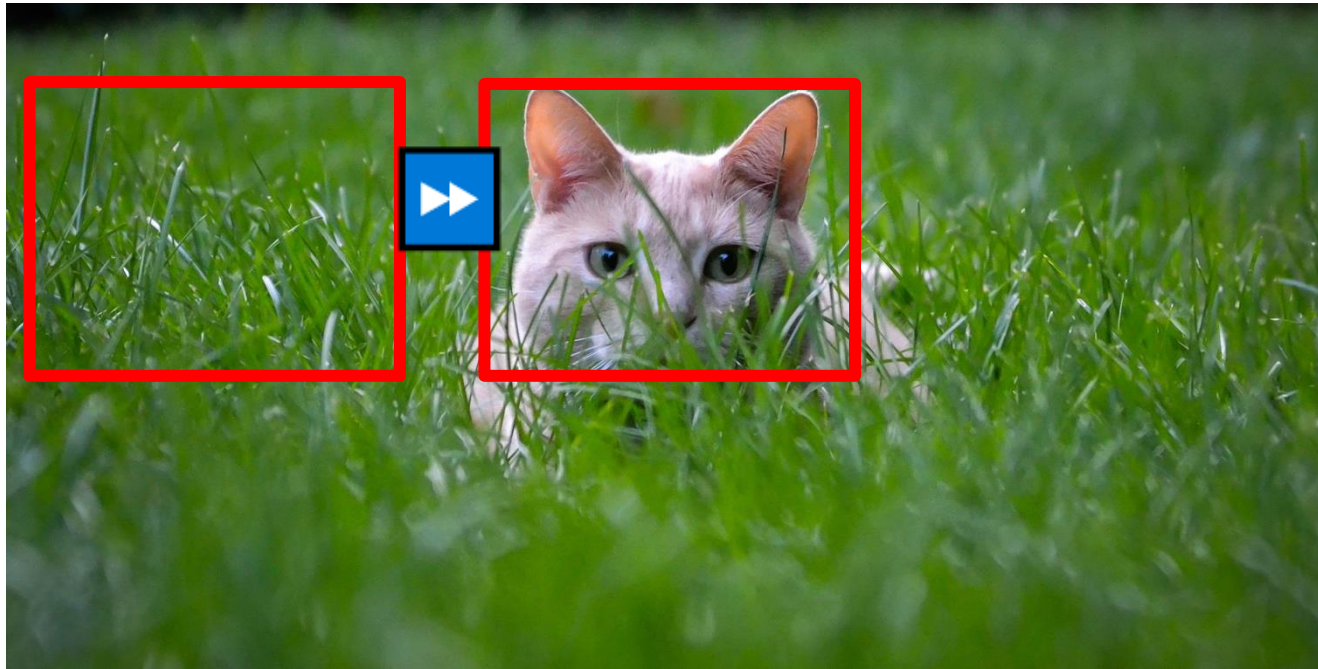


$$\begin{aligned} h_1 &= \sum_{|p| \leq \Delta} \mathbf{f}(p) \mathbf{x}(1 + p) \in 1^{\text{st}} \text{ patch} \\ h_2 &= \sum_{|p| \leq \Delta} \mathbf{f}(p) \mathbf{x}(2 + p) \in 2^{\text{nd}} \text{ patch} \\ &\quad \dots \end{aligned}$$

$$h_i = \sum_{|p| \leq \Delta} \mathbf{f}(p) \mathbf{x}(i + p) \in i^{\text{th}} \text{ patch}$$

local patch를 통해 시각 정보 처리

02. CNN – Convolution Layer



$$\begin{aligned} h_1 &= \sum_{|p| \leq \Delta} \mathbf{f}(p) \mathbf{x}(1 + p) \in 1^{\text{st}} \text{ patch} \\ h_2 &= \sum_{|p| \leq \Delta} \mathbf{f}(p) \mathbf{x}(2 + p) \in 2^{\text{nd}} \text{ patch} \\ &\dots \\ h_i &= \sum_{|p| \leq \Delta} \mathbf{f}(p) \mathbf{x}(i + p) \in i^{\text{th}} \text{ patch} \end{aligned}$$

filters

동일 parameter의 필터를 모든 패치에 대해 적용

02. CNN – Convolution Layer

Convolution 이란?

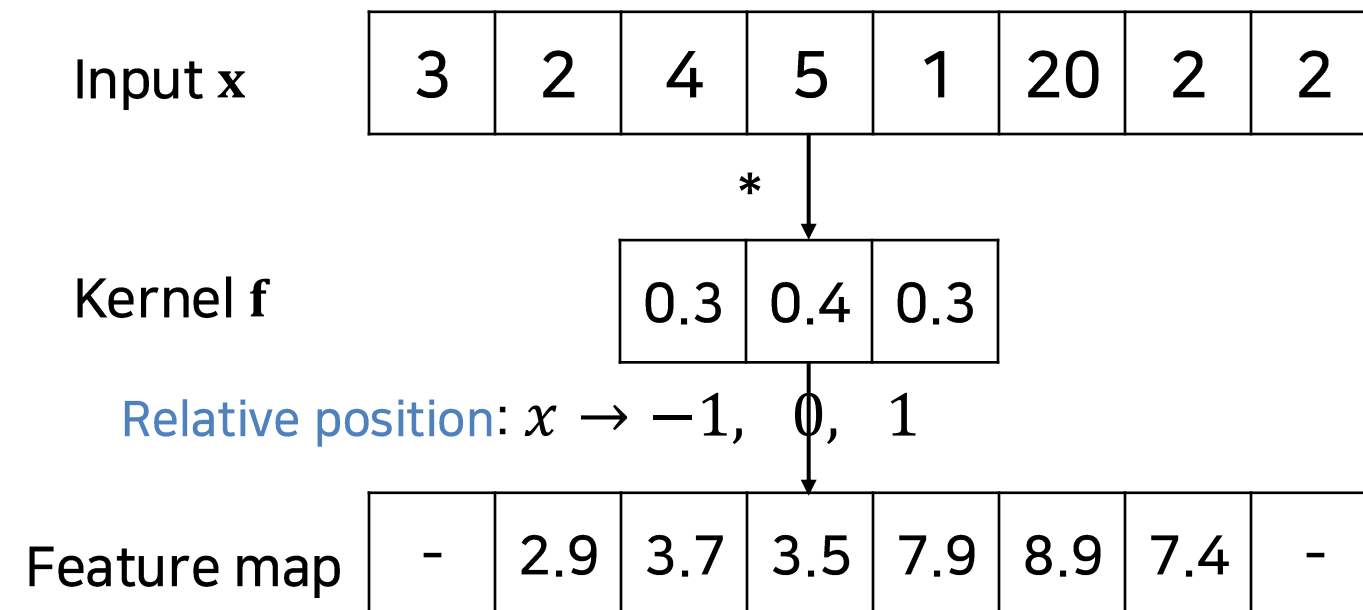
$\mathbf{x}$ 가 input, $\mathbf{f}$ 가 kernel

1D Conv $[\mathbf{f} * \mathbf{x}](i) = \sum_p \mathbf{f}(p) \mathbf{x}(i + p)$

2D Conv $[\mathbf{f} * \mathbf{x}](i, j) = \sum_{p, q} \mathbf{f}(p, q) \mathbf{x}(i + p, j + q)$

3D Conv $[\mathbf{f} * \mathbf{x}](i, j, k) = \sum_{p, q, r} \mathbf{f}(p, q, r) \mathbf{x}(i + p, j + q, k + r)$

(a) 1D Convolution



02. CNN – Convolution Layer

Convolution 이란?

$\mathbf{x}$ 가 input, $\mathbf{f}$ 가 kernel

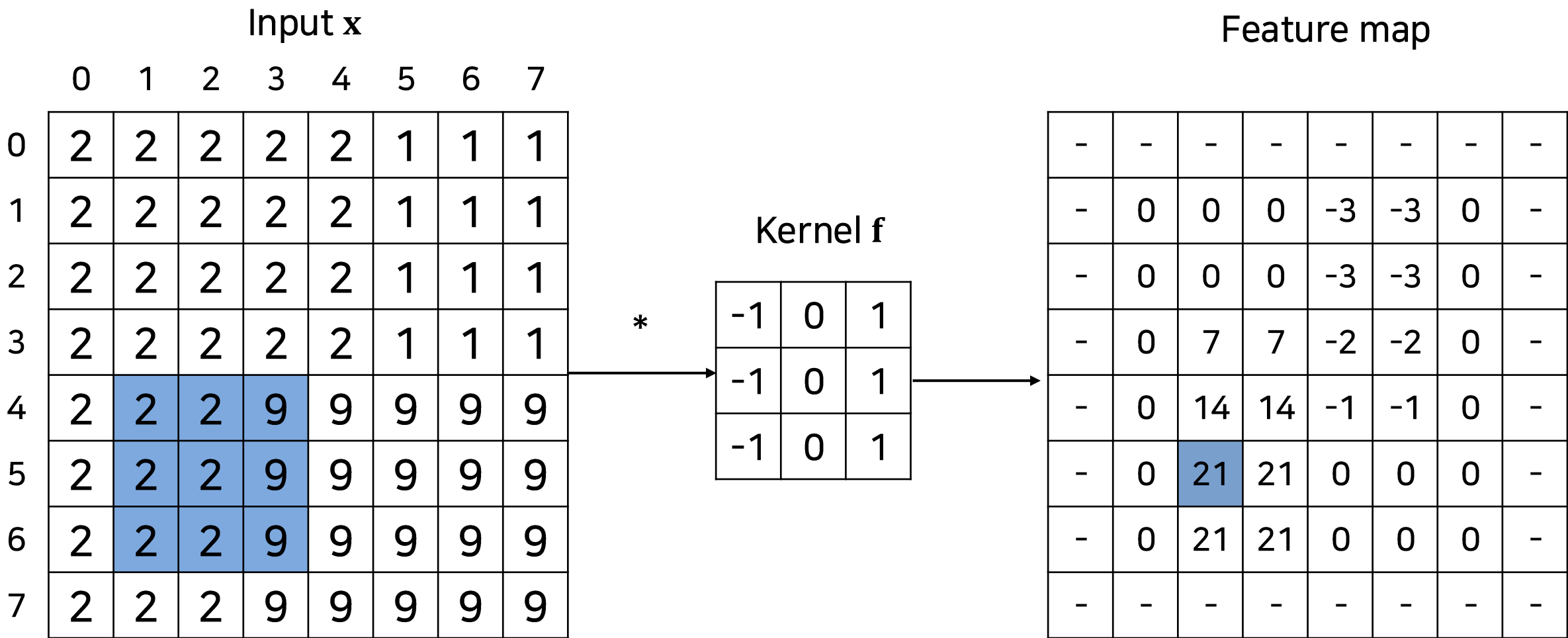
1D Conv $[\mathbf{f} * \mathbf{x}](i) = \sum_p \mathbf{f}(p)\mathbf{x}(i + p)$

2D Conv $[\mathbf{f} * \mathbf{x}](i, j) = \sum_{p, q} \mathbf{f}(p, q)\mathbf{x}(i + p, j + q)$

3D Conv $[\mathbf{f} * \mathbf{x}](i, j, k) = \sum_{p, q, r} \mathbf{f}(p, q, r)\mathbf{x}(i + p, j + q, k + r)$

(b) 2D Convolution

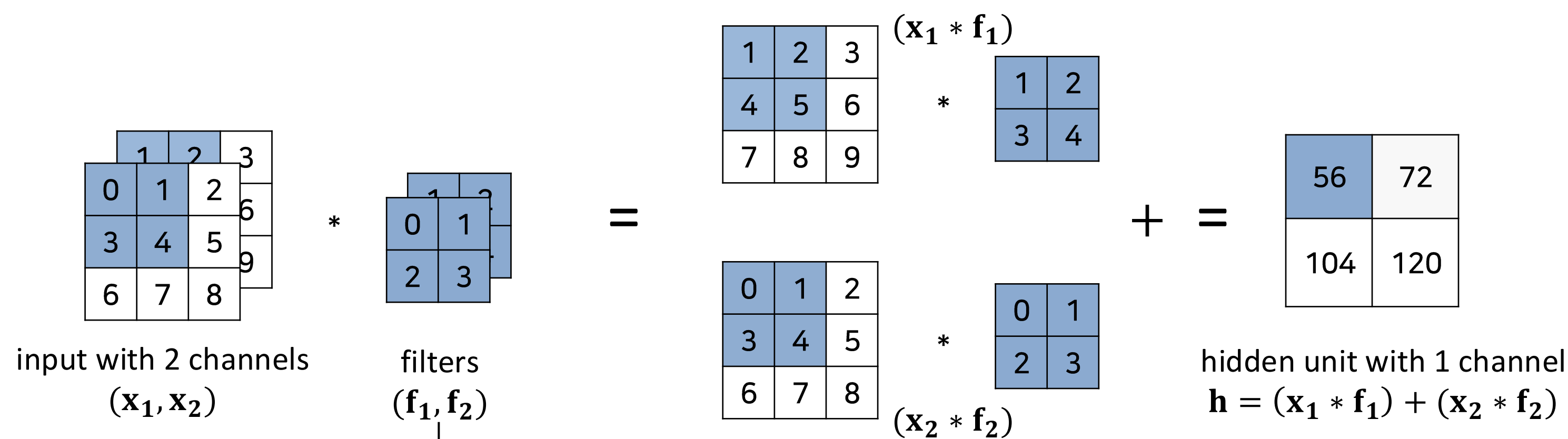
Relative position: $(i, j) \rightarrow (-1, -1)(-1, 0)(-1, 1)$
 $(0, -1)(0, 0)(0, 1)$
 $(1, -1)(1, 0)(1, 1)$



02. CNN – Convolution Layer

Convolution 이란?

- Applying to multiple **input** channels (Tensor)
- : 이미지는 주로 multiple channel로 구성 (e.g., RGB color image!)



element-wise multiplication

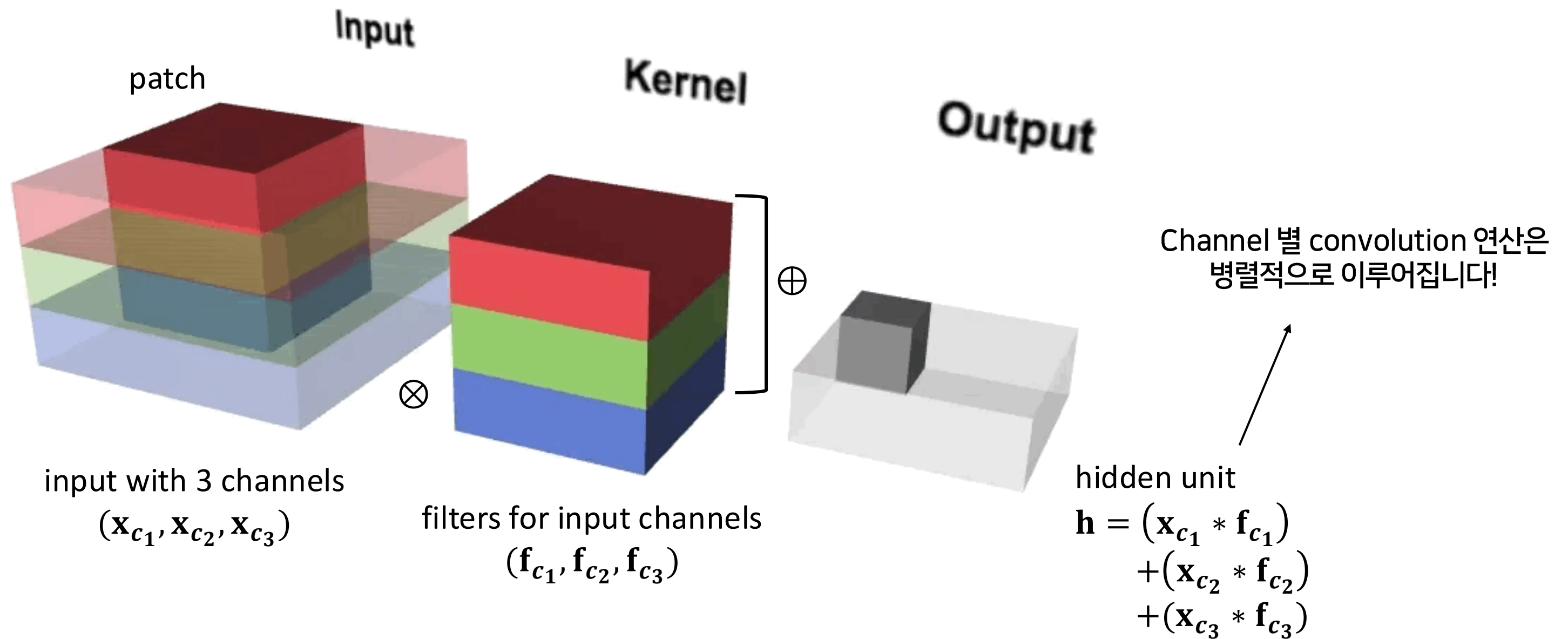
$$\begin{pmatrix} 1 & 2 \\ 4 & 5 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 3 & 4 \end{pmatrix} \odot \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 2 & 3 \end{pmatrix} = 56$$

$x_1 \quad x_2 \quad f_1 \quad f_2$

02. CNN – Convolution Layer

Convolution 이란?

- Applying to multiple **input** channels (Tensor)



02. CNN – Convolution Layer

Convolution 이란?

- Filter의 역할 (<https://setosa.io/ev/image-kernels/>)



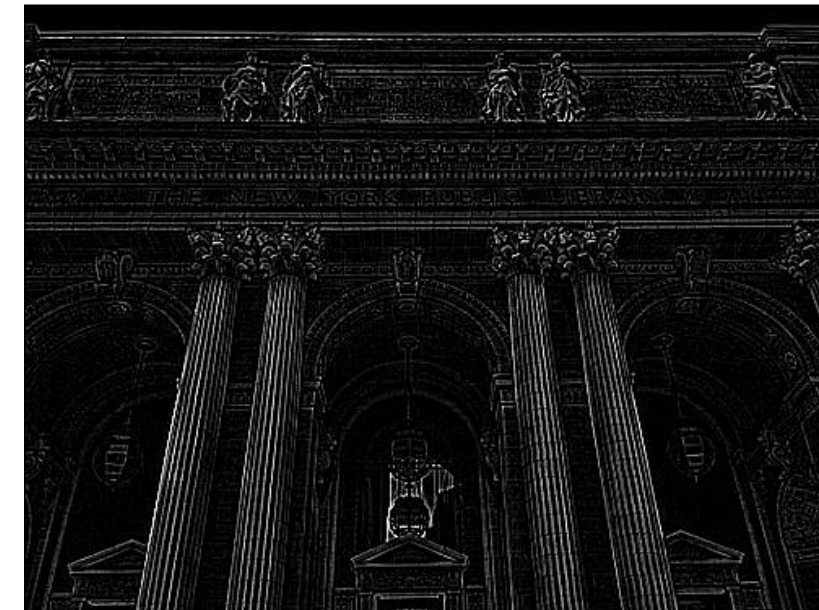
blur

0.06	0.12	0.06
0.12	0.25	0.12
0.06	0.12	0.06



outline

-1	-1	-1
-1	8	-1
-1	-1	-1



02. CNN – Convolution Layer

Convolution 이란?

- Filter의 역할 (<https://setosa.io/ev/image-kernels/>)

** MLP에서는 hidden unit의 개수를 늘려서 표현력을 늘릴 수 있음.

→ Q. Convolution 연산에서는?

- filter size (3x3, 5x5 ...) X
- filter depth (multiple input channels) X

Depending on the element values, a kernel can cause a wide range of effects:

Operation	Kernel ω	Image result $g(x,y)$
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Ridge or edge detection	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur 3 × 3 (approximation)	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	
Gaussian blur 5 × 5 (approximation)	$\frac{1}{256} \begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$	

02. CNN – Convolution Layer

Convolution 이란?

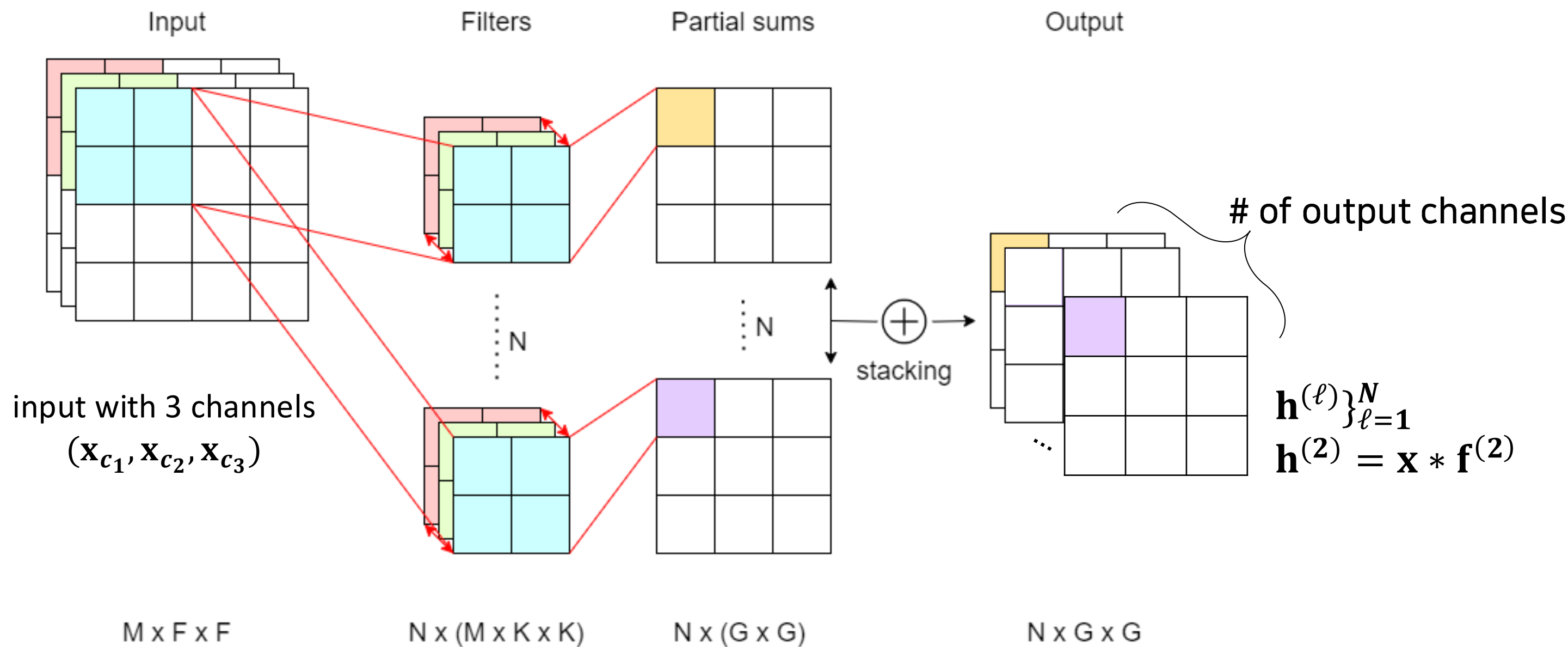
- Convolution with multiple **output** channels
- : 같은 input 이미지에 대해 다른 filter 적용하면 다른 feature map 이 나옴.
- : multiple filter를 사용해 multiple output channel 획득 가능!



02. CNN – Convolution Layer

Convolution 이란?

- Convolution with multiple **output** channels



multiple filters
 $\{(\mathbf{f}_{c_1}^{(\ell)}, \mathbf{f}_{c_2}^{(\ell)}, \mathbf{f}_{c_3}^{(\ell)})\}_{\ell=1}^N$

→ how many filters do we need?

02. CNN – Convolution Layer

Convolution 이란?

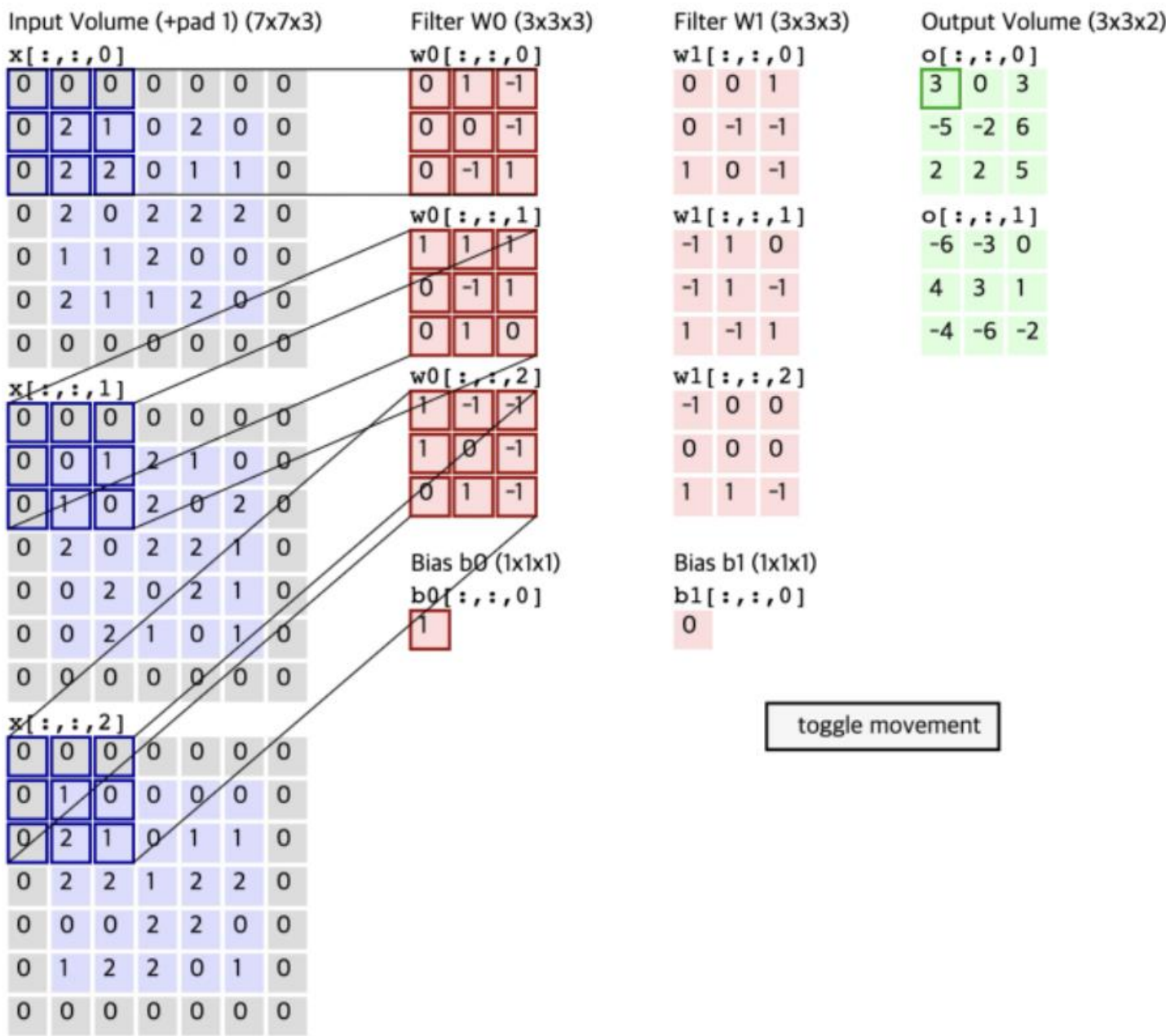
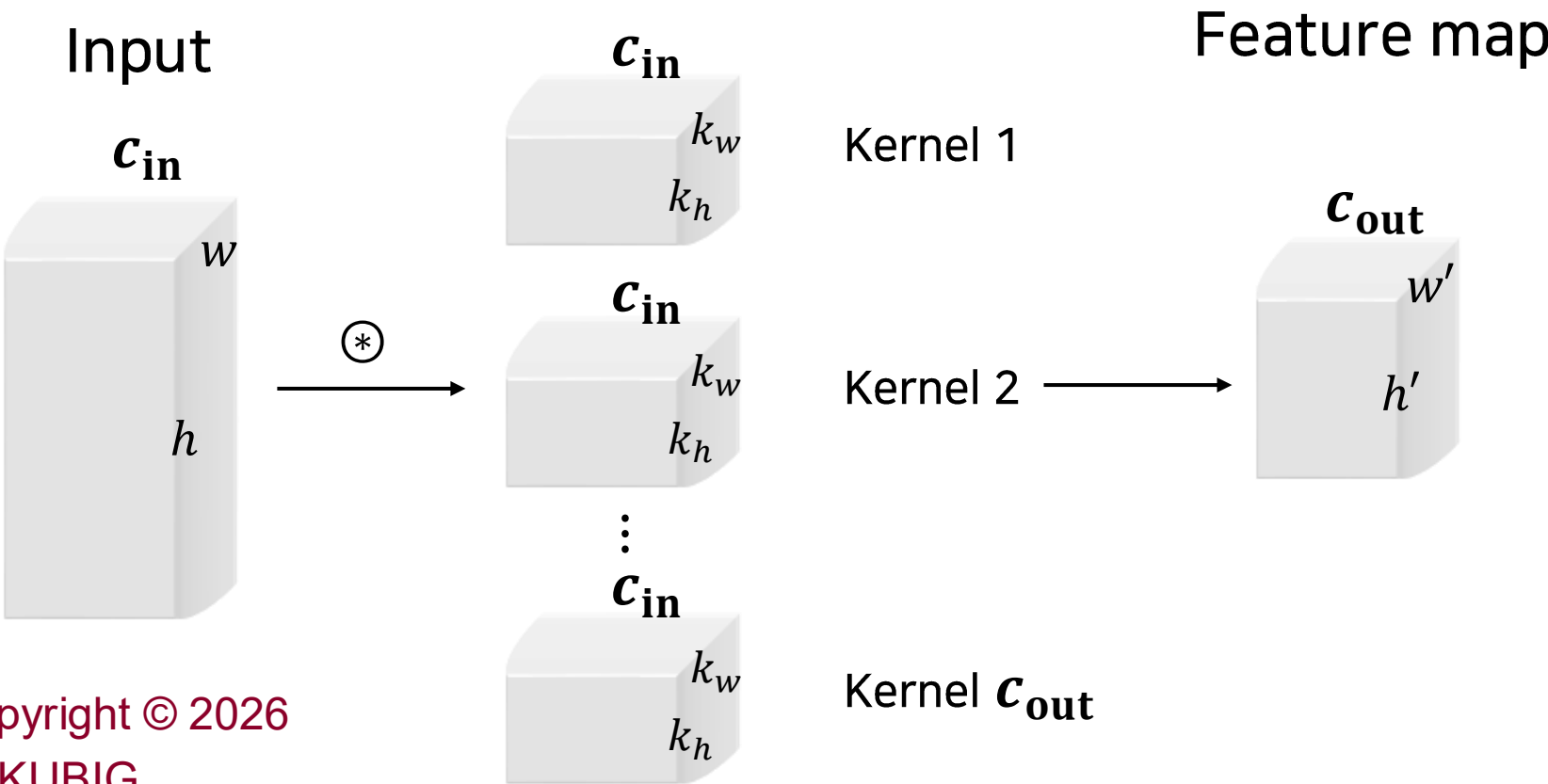
- Parameter size of convolutional net

c_{in} : number of input channels

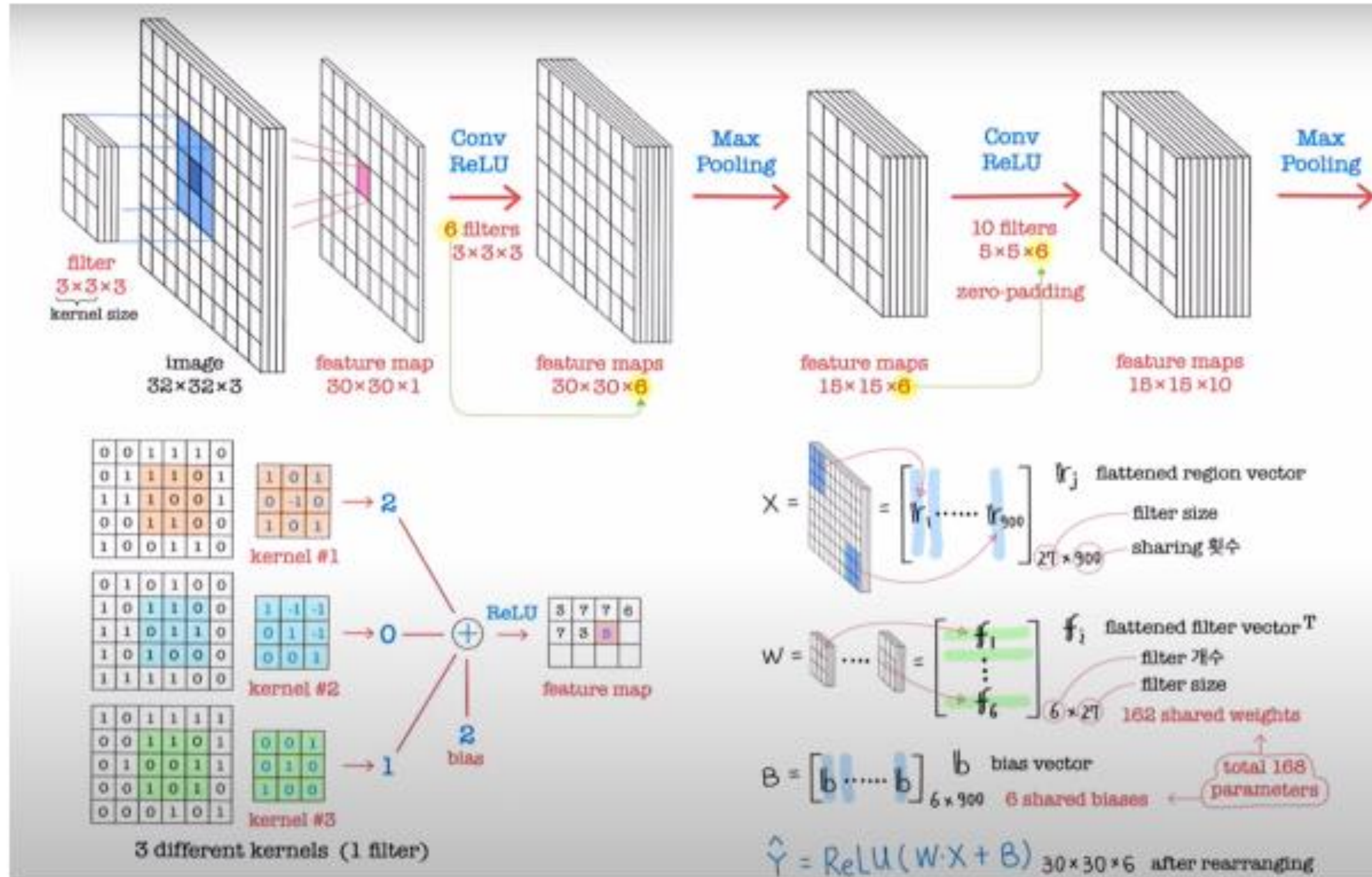
c_{out} : number of output channels

k_h, k_w : height and width of filter

Parameter size: $c_{out} \times c_{in} \times k_h \times k_w$
combination of input-output channels filter size



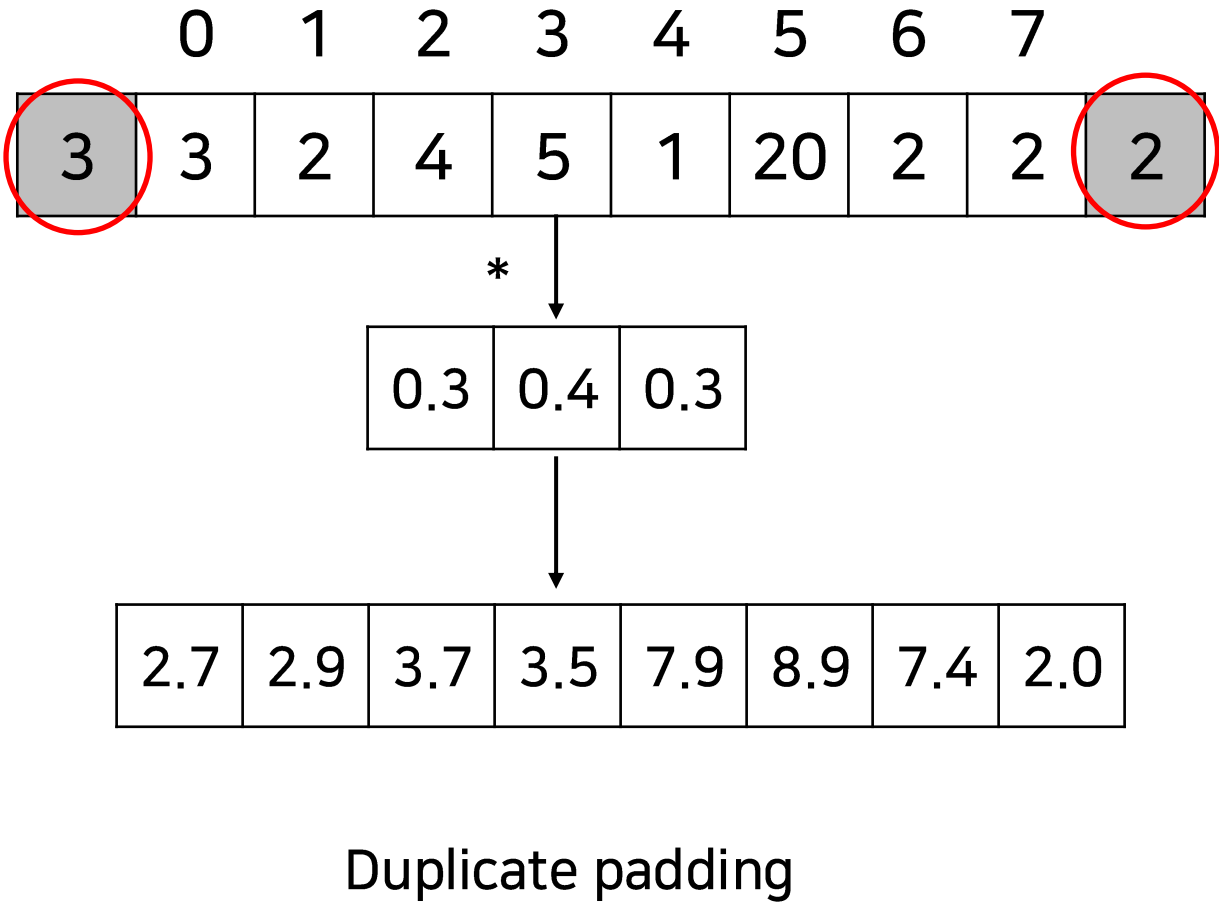
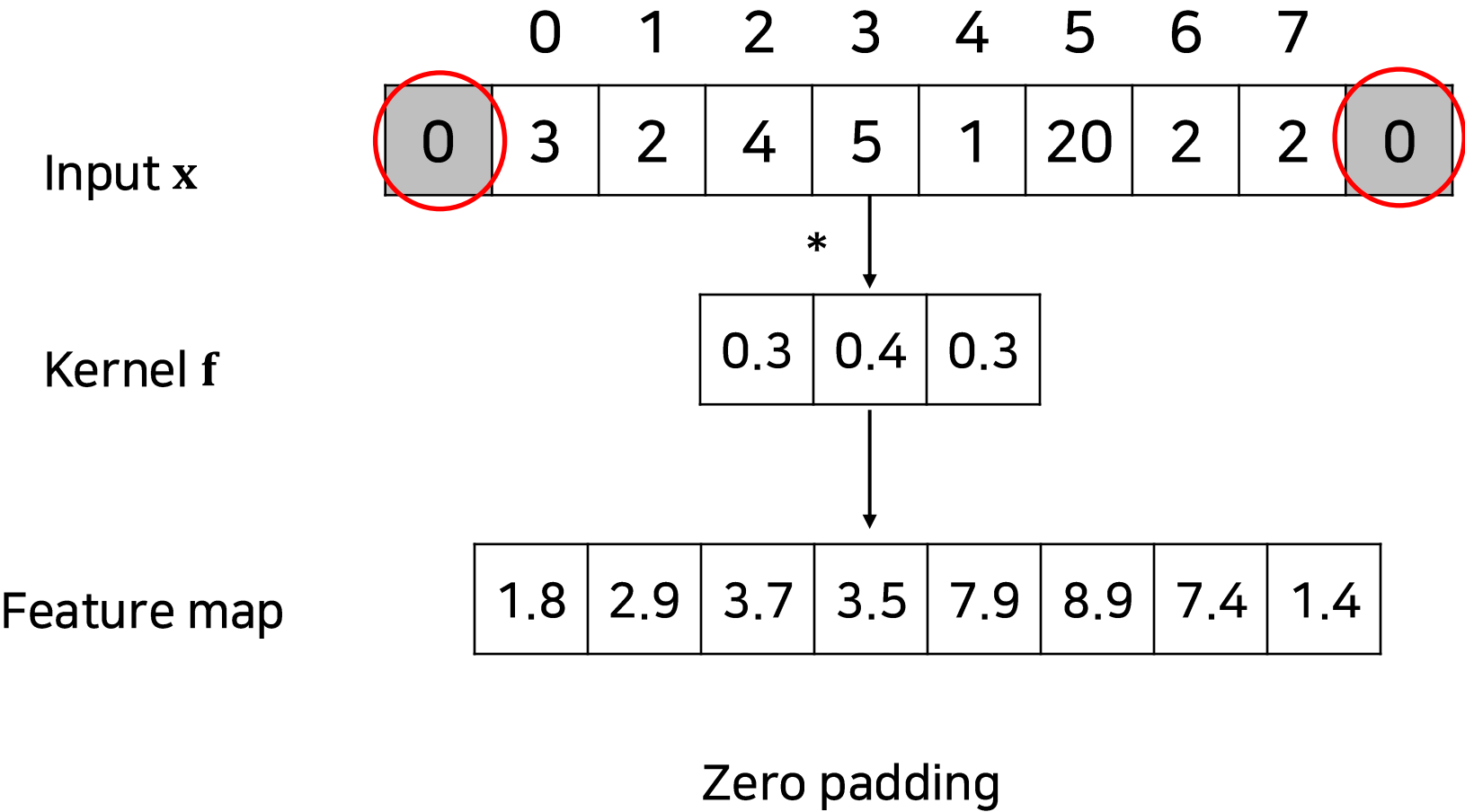
02. CNN – Convolution Layer



02. CNN – Convolution Layer

Padding & Stride

- Padding
- : Padding은 input의 edge 부분이 잘리지 않도록 함.

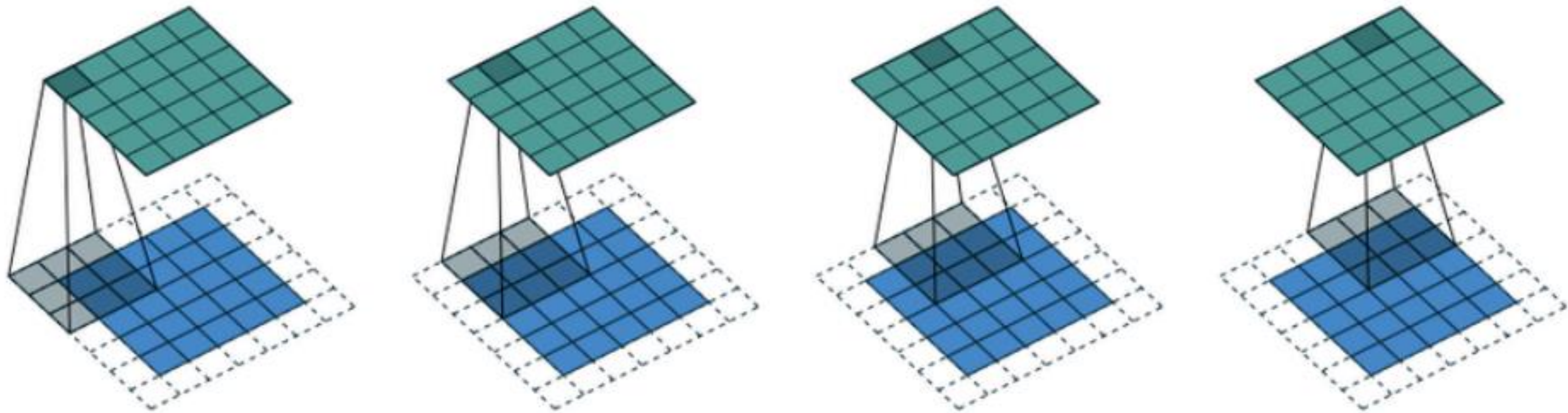


02. CNN – Convolution Layer

Padding & Stride

- Padding

: Padding은 input의 edge 부분이 잘리지 않도록 함.



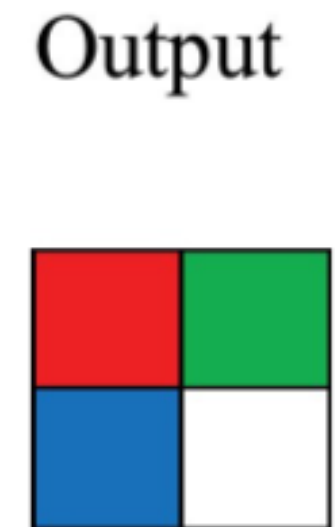
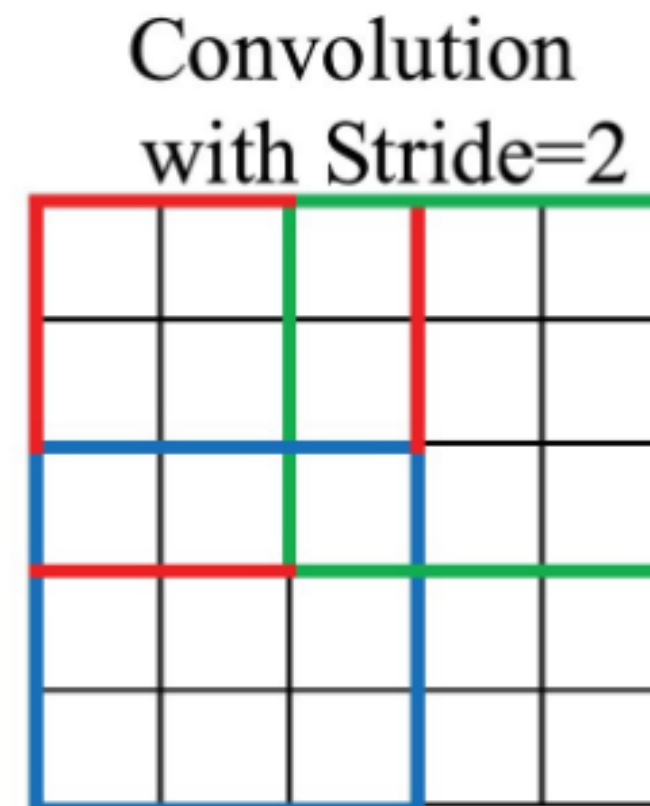
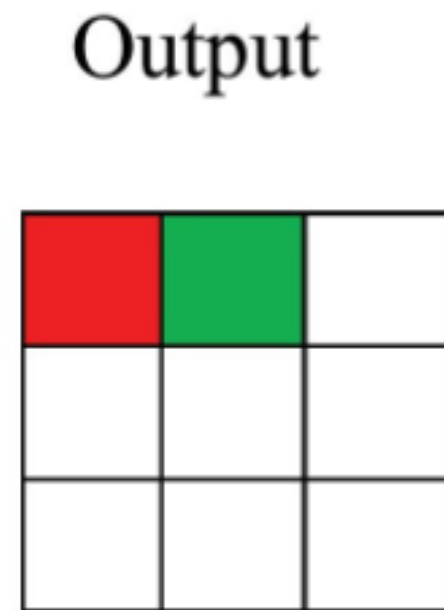
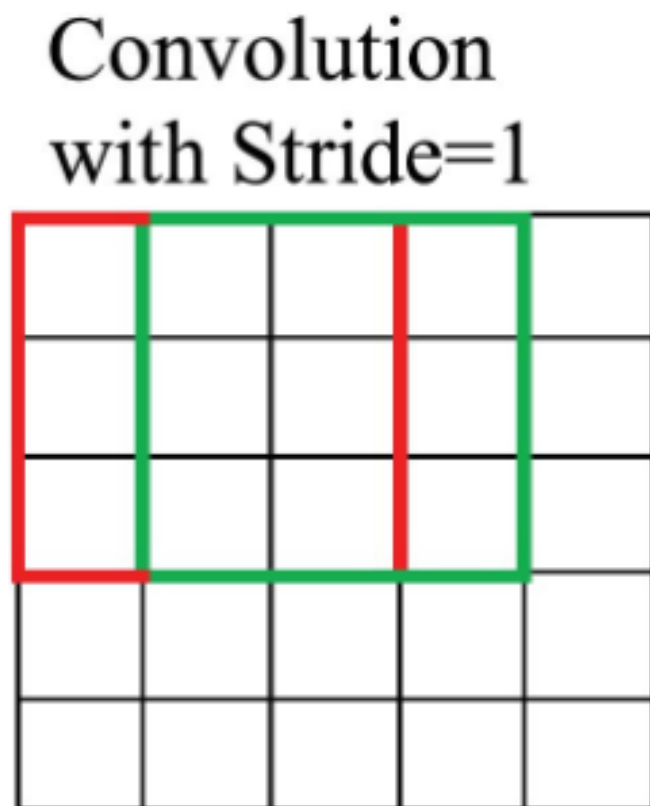
02. CNN – Convolution Layer

Padding & Stride

- Stride

: Stride를 키움으로써 feature map의 크기를 줄일 수 있음.

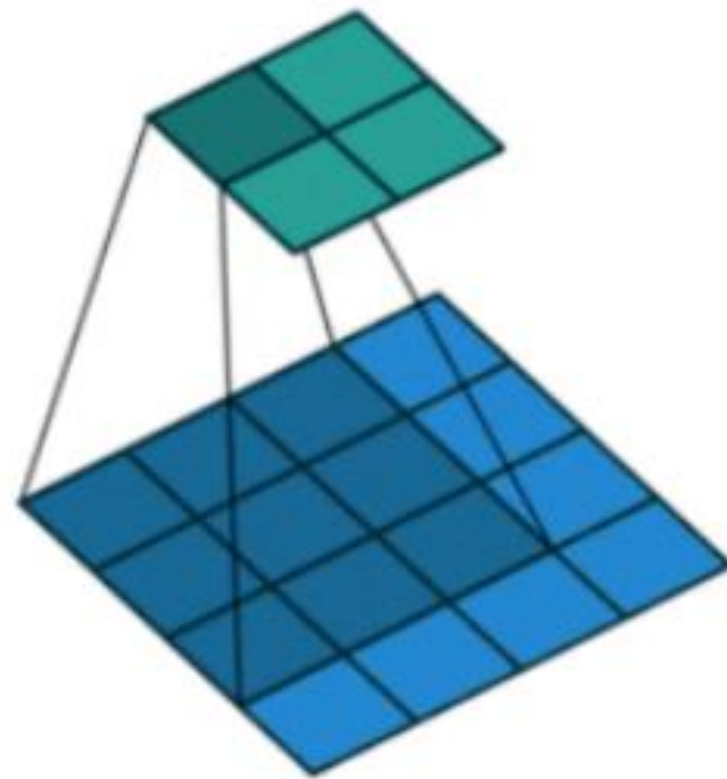
: Stride 이 k 면, kernel을 k 간격으로 적용 → 2D 이미지에서 feature map 은 $1/k^2$ 크기로 감소



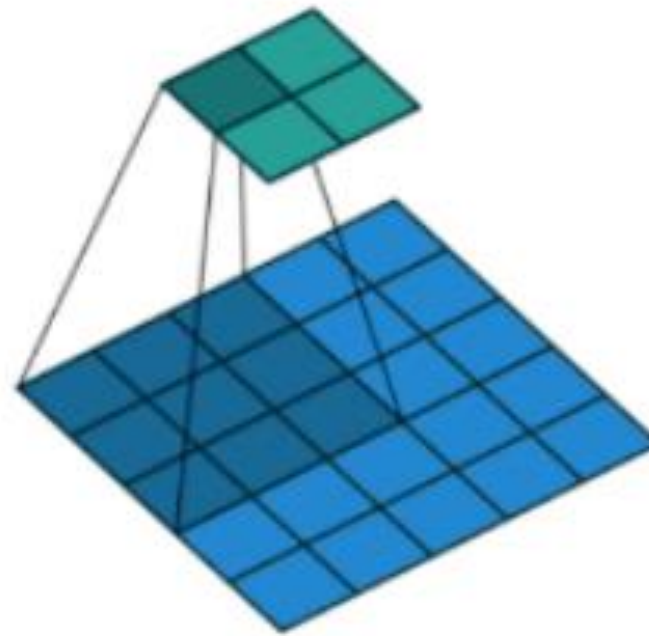
- If stride=1, feature map 크기는 original map과 동일
- If stride=2, feature map 크기는 original map의 1/4

02. CNN – Convolution Layer

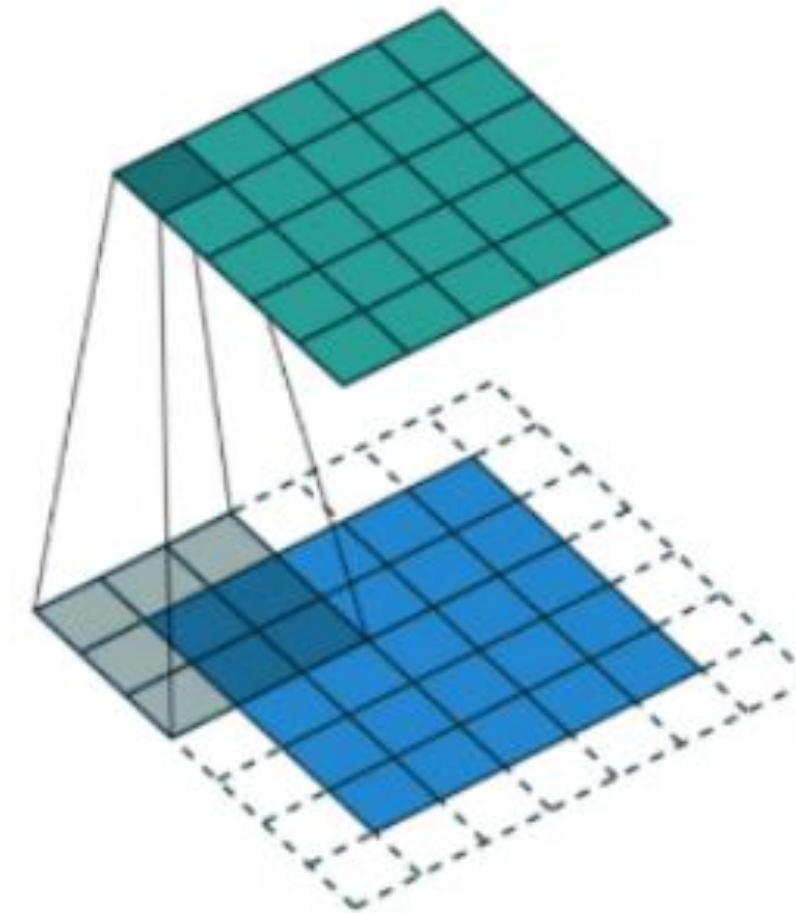
Padding & Stride



Vanilla convolution



convolution + stride



convolution + padding

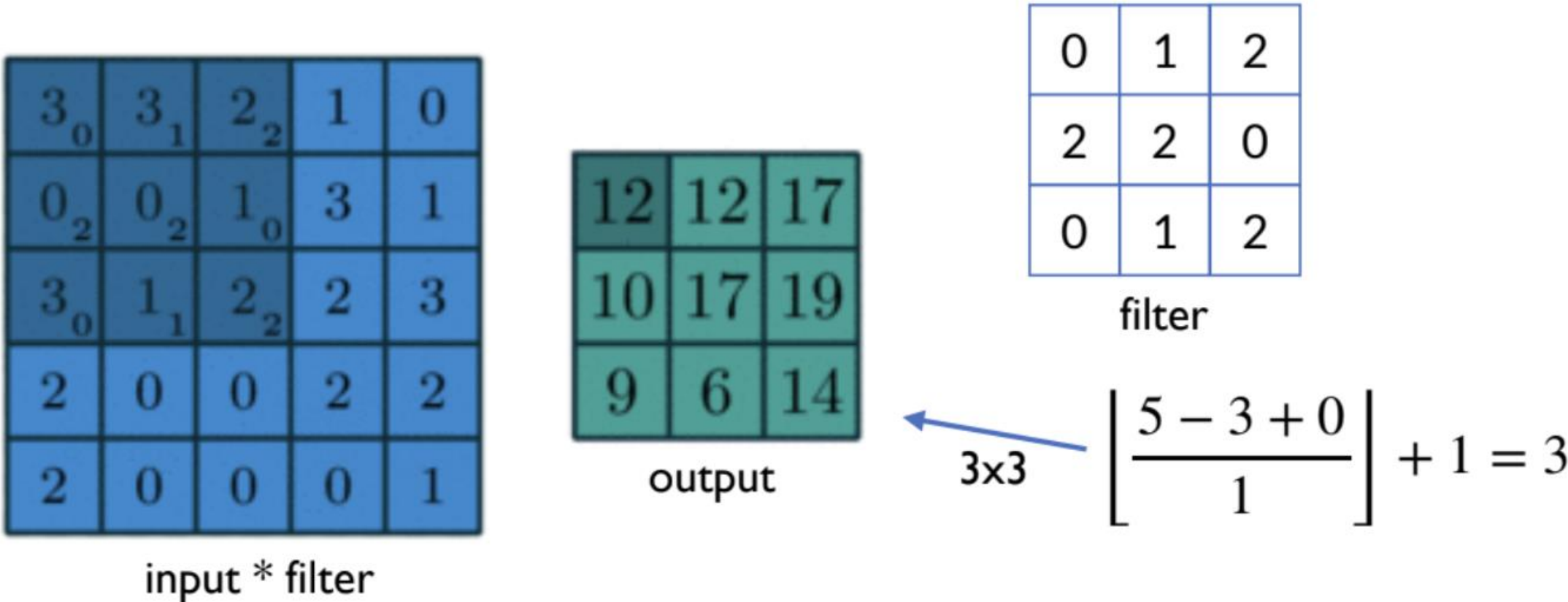
(k_h, k_w) filter size (p_h, p_w) padding size (s_h, s_w) stride size

$$(h_{\text{out}}, w_{\text{out}}) = \left(\left\lfloor \frac{h_{\text{in}} - k_h + 2p_h}{s_h} \right\rfloor + 1, \left\lfloor \frac{w_{\text{in}} - k_w + 2p_w}{s_w} \right\rfloor + 1 \right)$$

02. CNN – Convolution Layer

Padding & Stride

Vanilla 3x3 convolution



(k_h, k_w) filter size (p_h, p_w) padding size (s_h, s_w) stride size

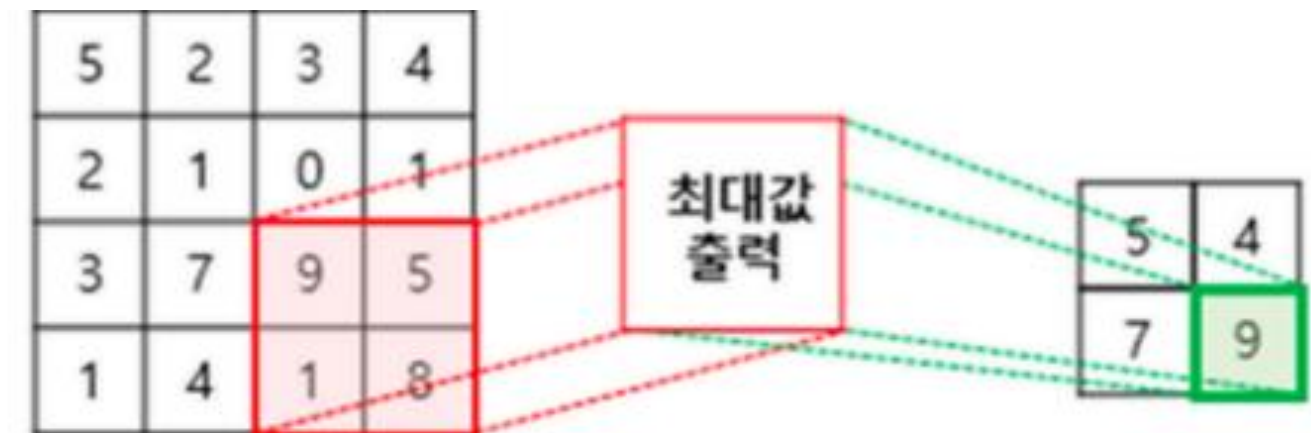
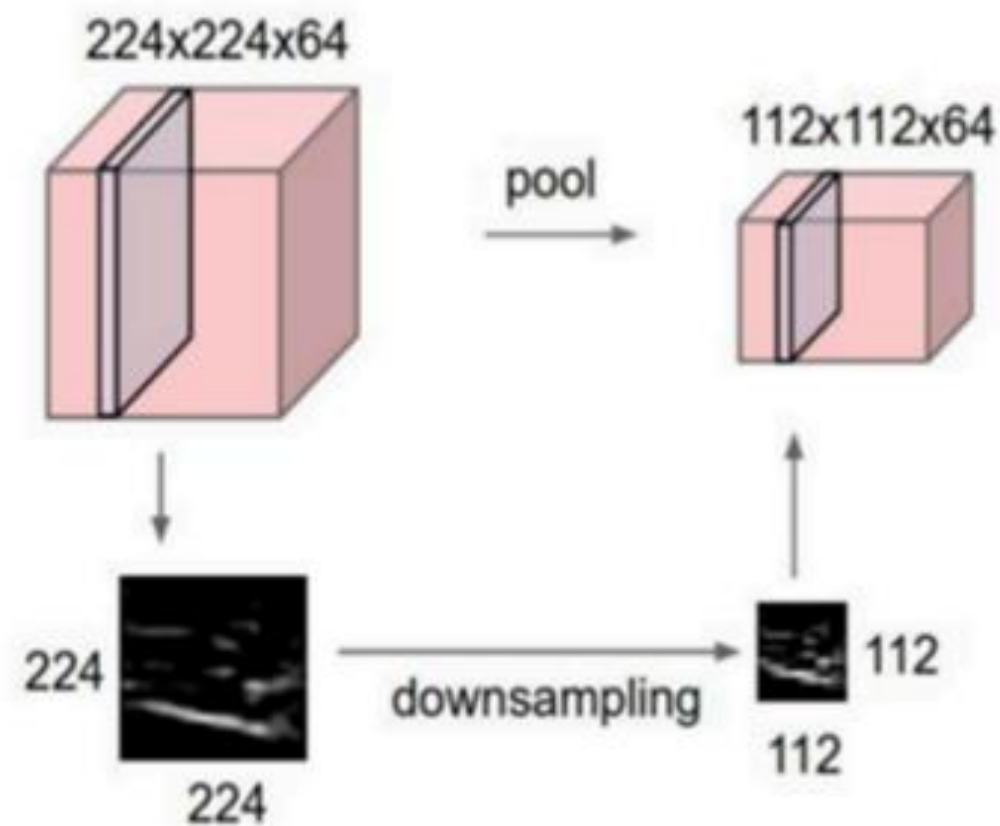
$$(h_{out}, w_{out}) = \left(\left\lfloor \frac{h_{in} - k_h + 2p_h}{s_h} \right\rfloor + 1, \left\lfloor \frac{w_{in} - k_w + 2p_w}{s_w} \right\rfloor + 1 \right)$$

02. CNN – Pooling Layer

Pooling Layer

- Pooling: 점진적으로 spatial resolution을 줄여줌.
- : aggregate information
- : downsampling
- : parameter-free operator
- : local translation invariance

* Pooling은 학습 가능한 파라미터가 없습니다!



Pooling을 통해서 차원을 축소

Max Pooling, Mean Pooling

Conv 계산처럼 Stride가 존재

02. CNN – Pooling Layer

Pooling Layer

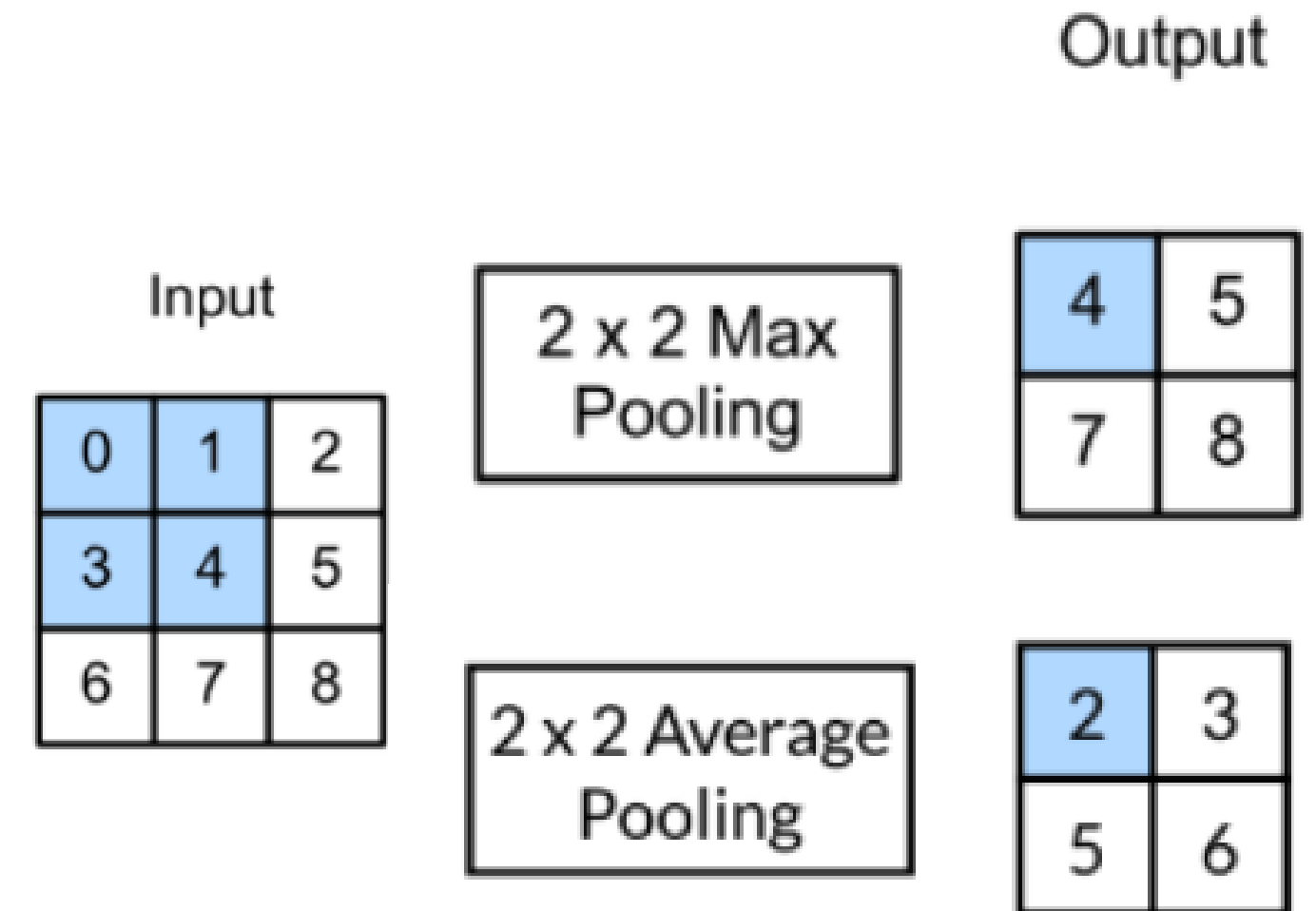
- Pooling의 종류

1. Maximum pooling

```
X = torch.tensor([[0.0, 1.0, 2.0], [3.0, 4.0, 5.0], [6.0, 7.0, 8.0]])  
pool2d(X, (2, 2))
```

2. Average pooling

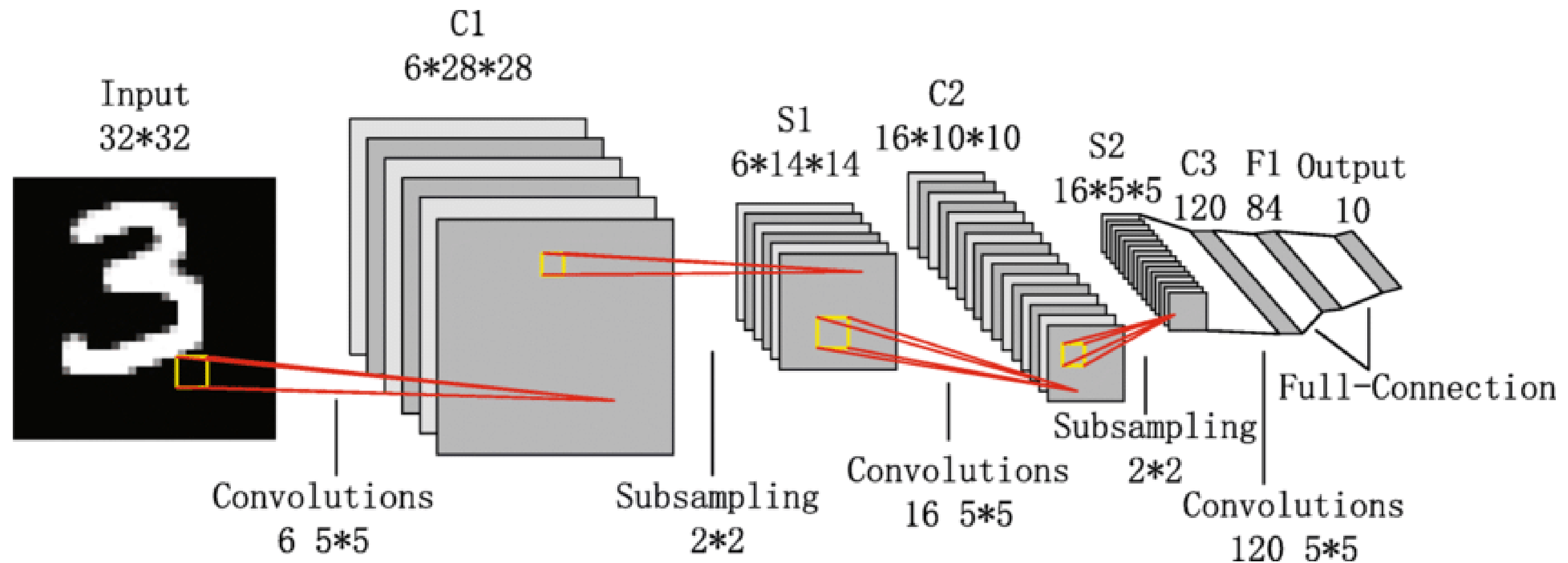
```
pool2d(X, (2, 2), 'avg')
```



02. CNN – Overall Structure

Overall Structure

- Example: LeNet-5



02. CNN

Parameter 개수 비교

시나리오

- 입력 이미지의 크기 : 32x32 픽셀
- 입력 이미지의 채널 수 : 3 (RGB)
- MLP의 hidden layer: 1개, 뉴런의 수는 128개
- CNN의 convolution layer: 32개 필터, 각 필터의 크기 3x3

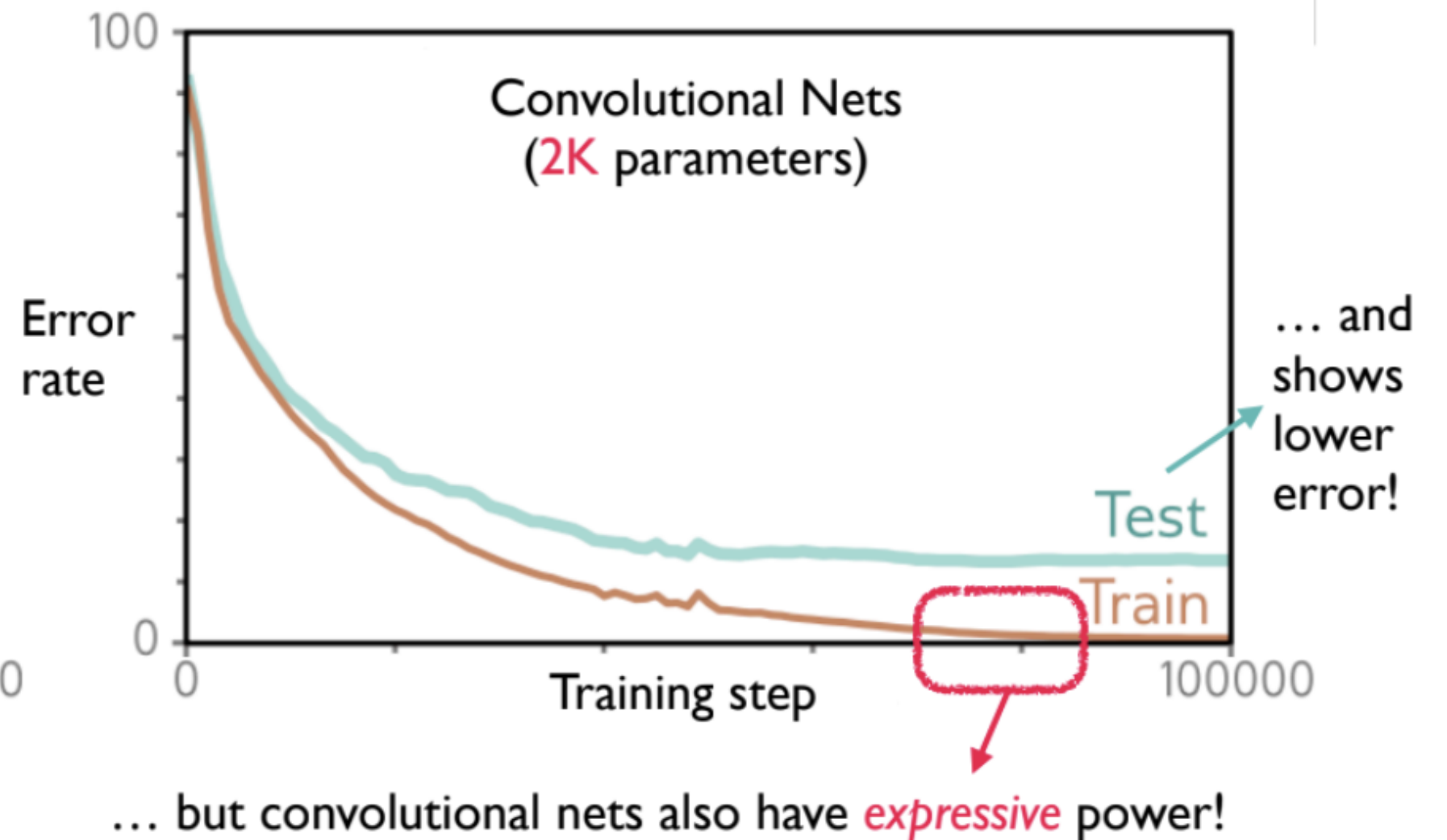
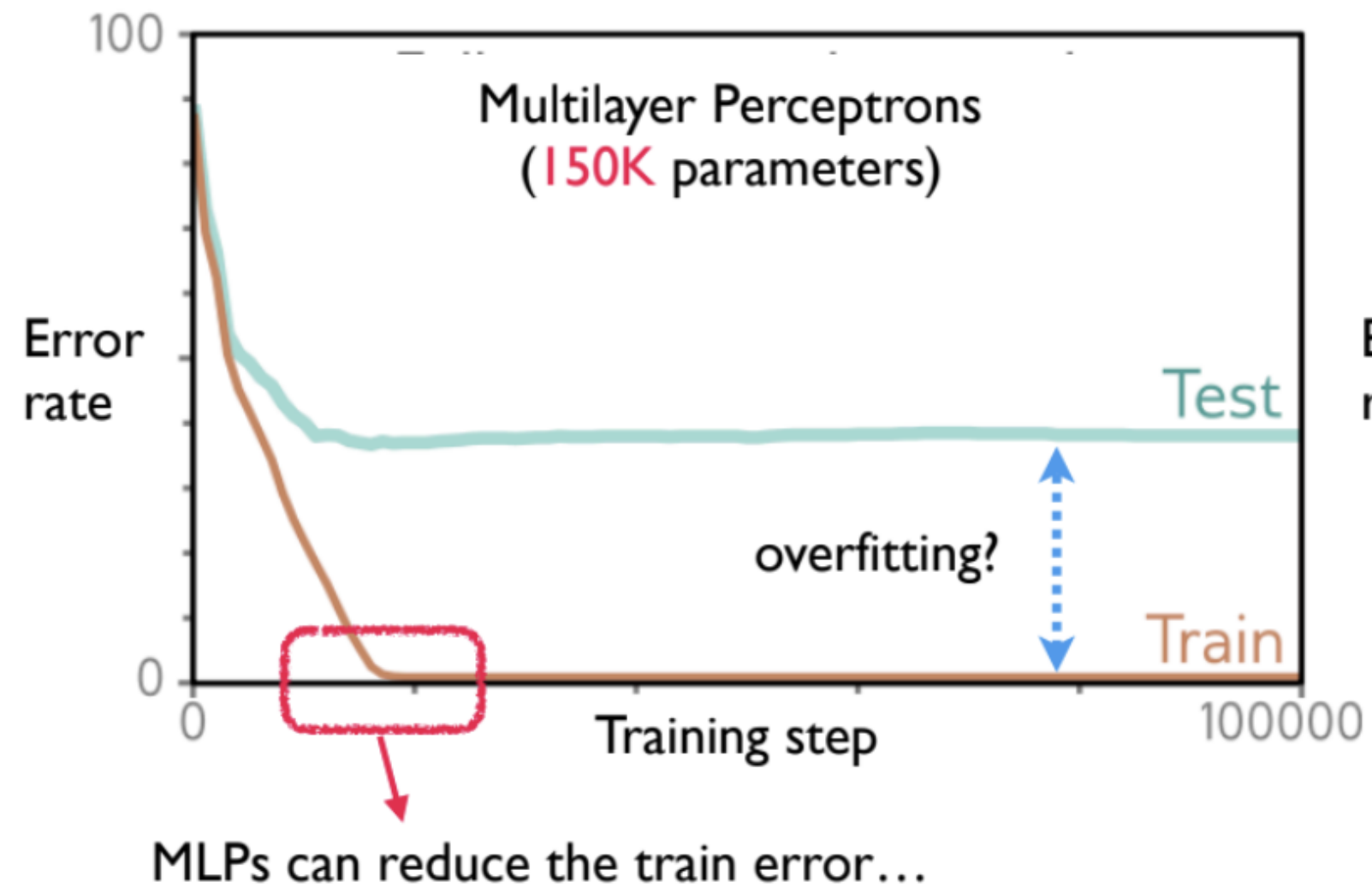
MLP의 파라미터 계산 :

CNN의 파라미터 계산 :

02. CNN

Example: MNIST

MLP도 MNIST에 대해 예측 성능이 높긴 하지만,
convolutional nets는 **더 적은 파라미터**로 효율적인 학습 가능



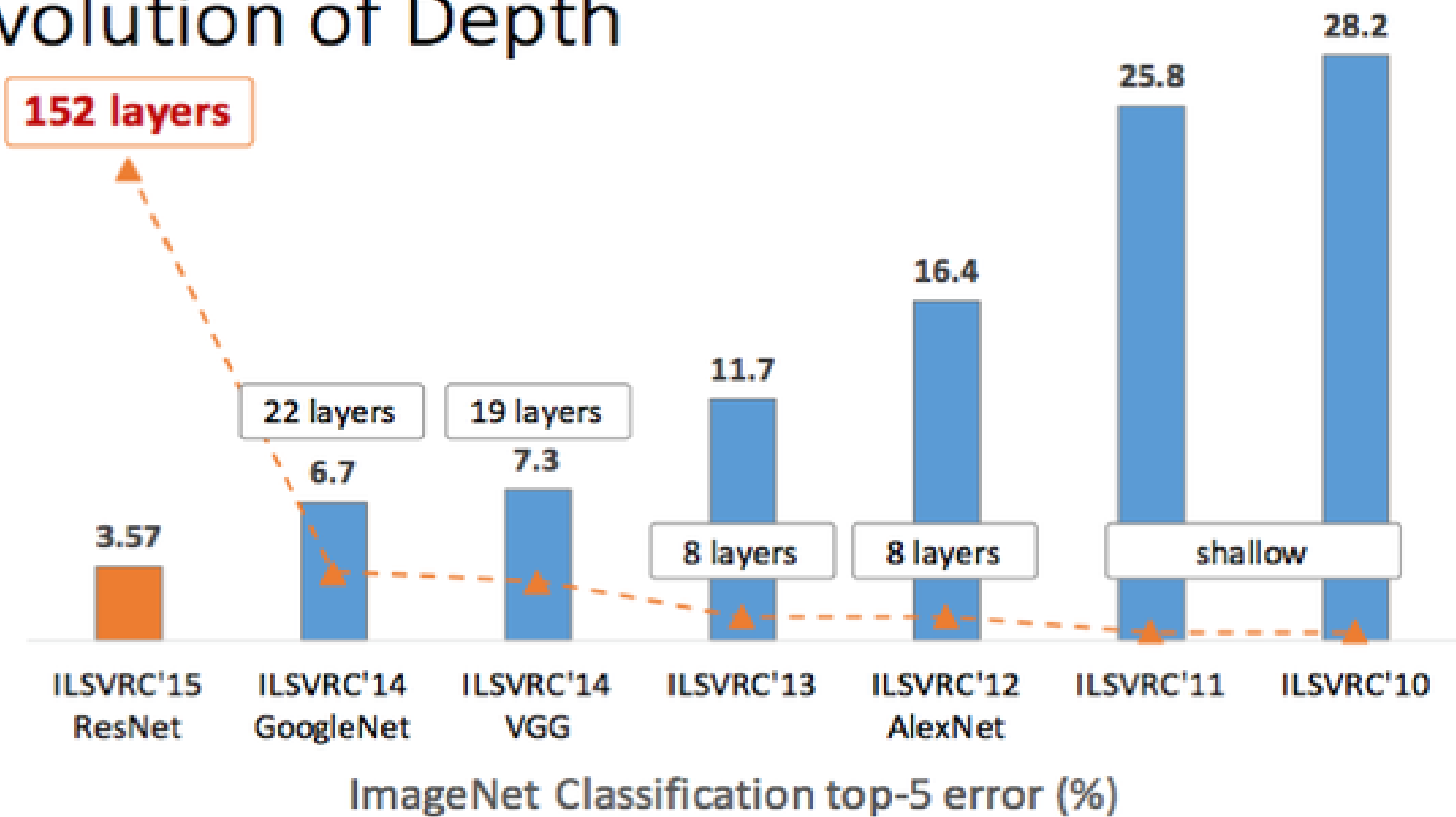
03

Deeper Layers

03. Deeper Layers

ImageNet classification challenge

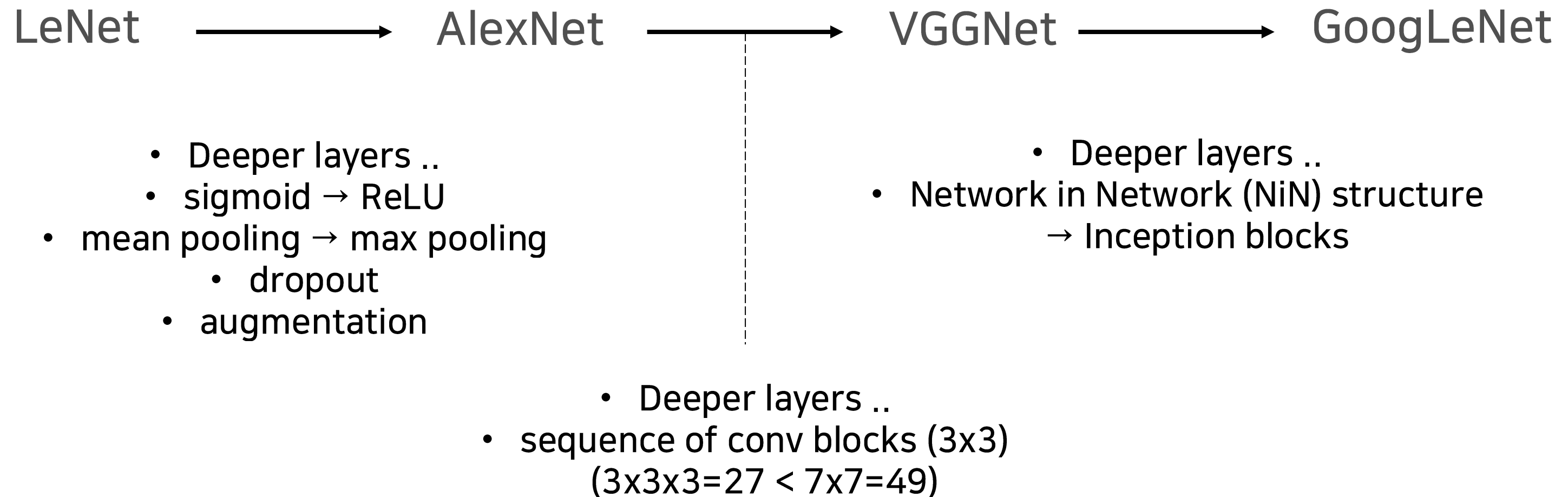
Revolution of Depth



03. Deeper Layers

ImageNet classification challenge

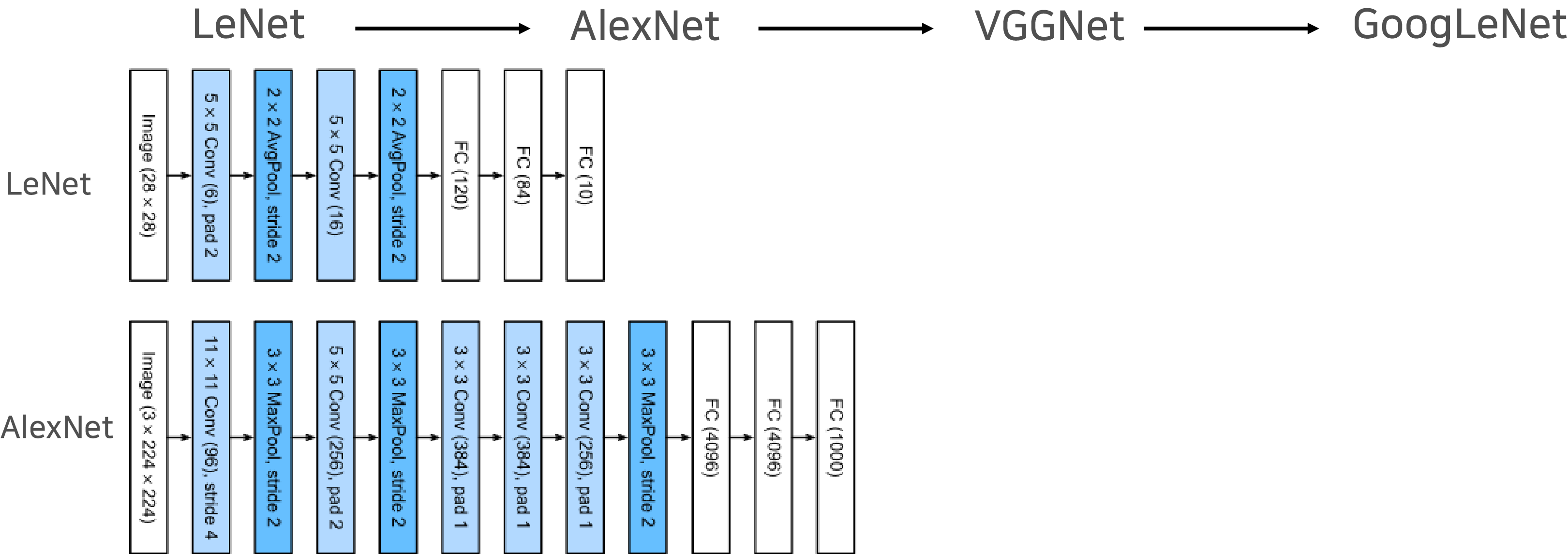
- 배경... 간략히 [자세한 내용은 d2l 참고 !]



03. Deeper Layers

ImageNet classification challenge

- 배경... 간략히



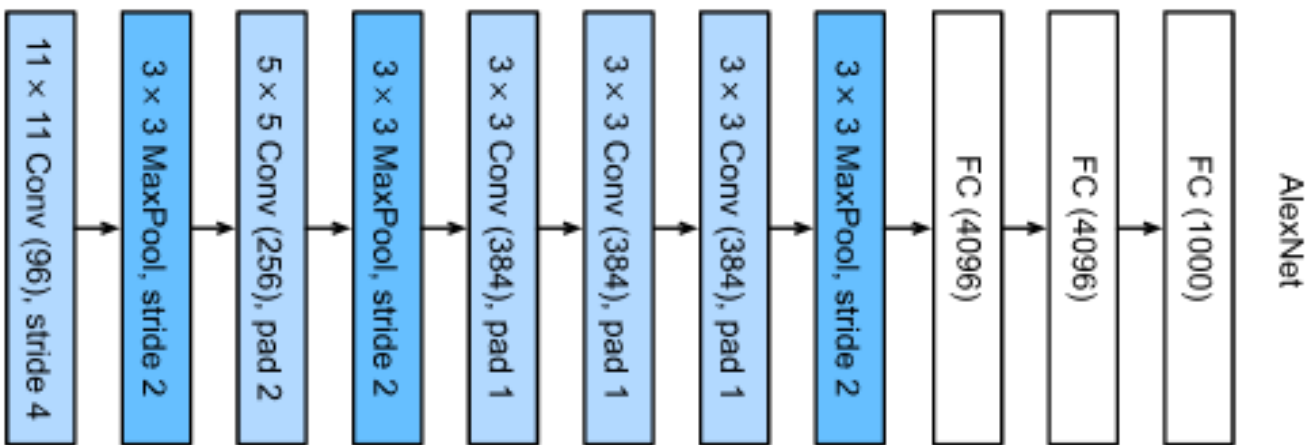
03. Deeper Layers

ImageNet classification challenge

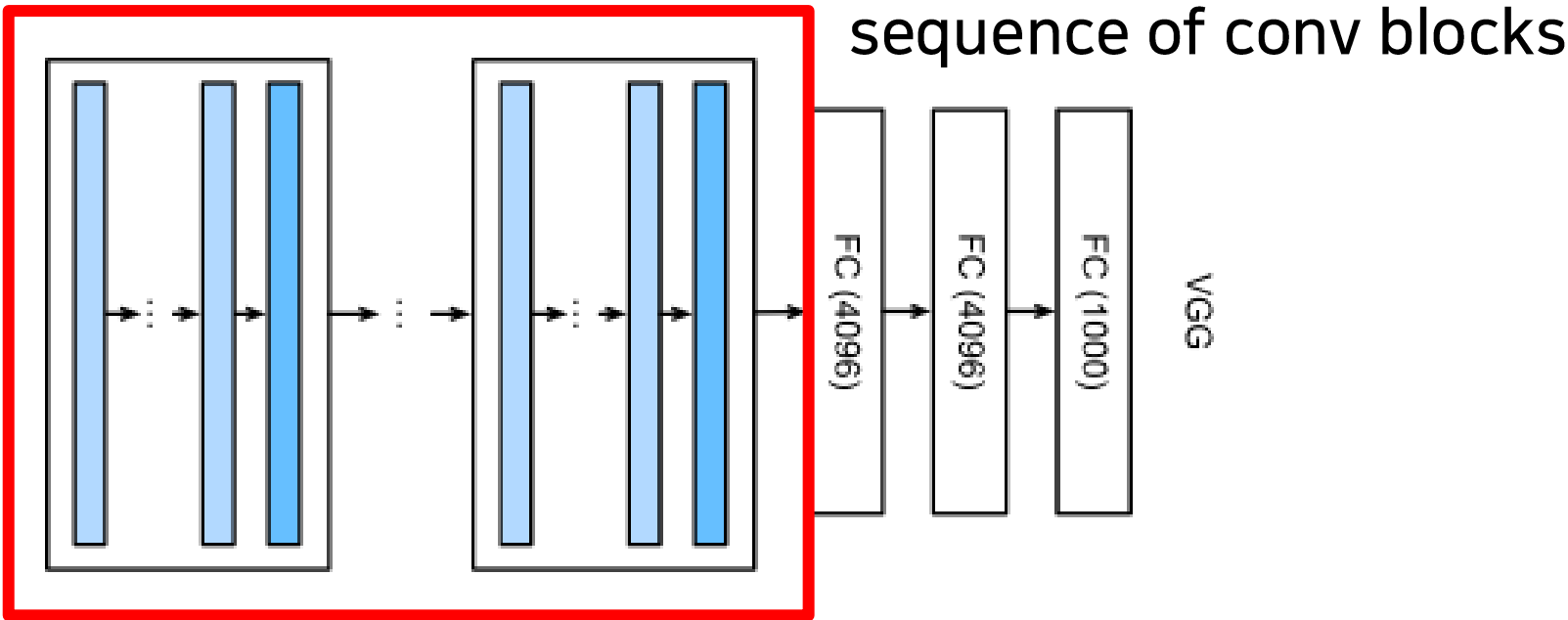
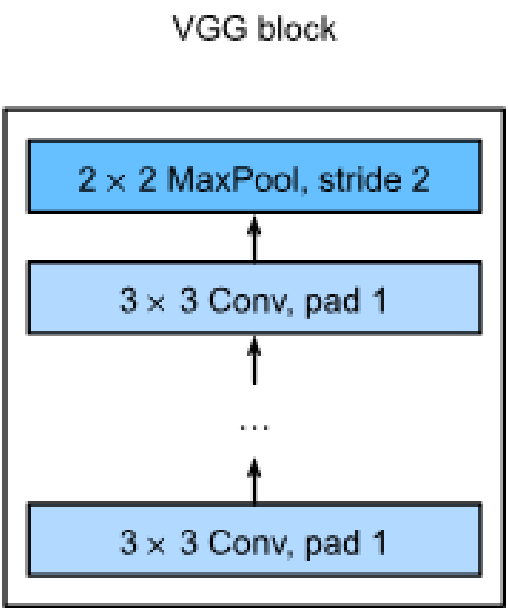
- 배경... 간략히

LeNet → AlexNet → VGGNet → GoogLeNet

AlexNet



VGGNet

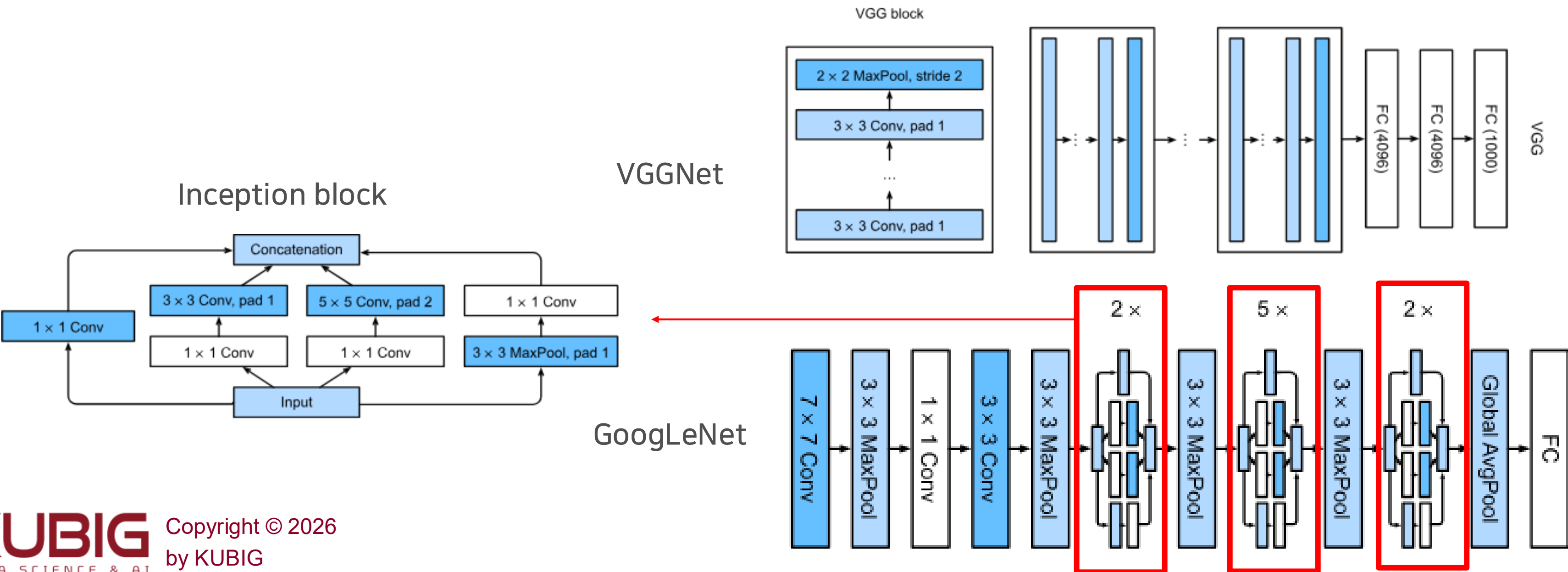


03. Deeper Layers

ImageNet classification challenge

- 배경... 간략히

LeNet → AlexNet → VGGNet → GoogLeNet



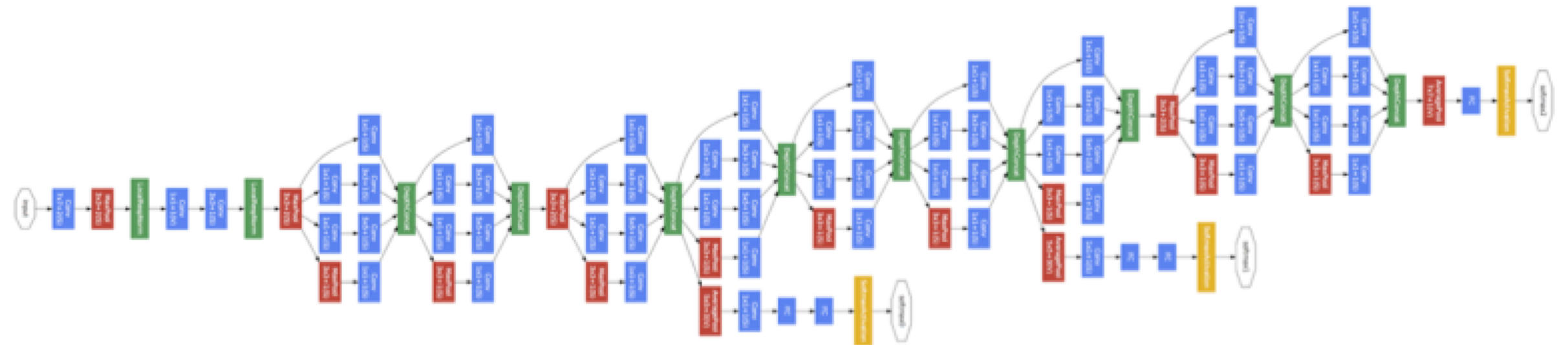
03. Deeper Layers

ImageNet classification challenge

- More layers do not imply more parameters * 알아만 두기!

VGGNet (19 layers) → GoogLeNet (22 layers)

GoogLeNet
(22 layers)



Convolution
Pooling
Softmax
Other

03. Deeper Layers

ImageNet classification challenge

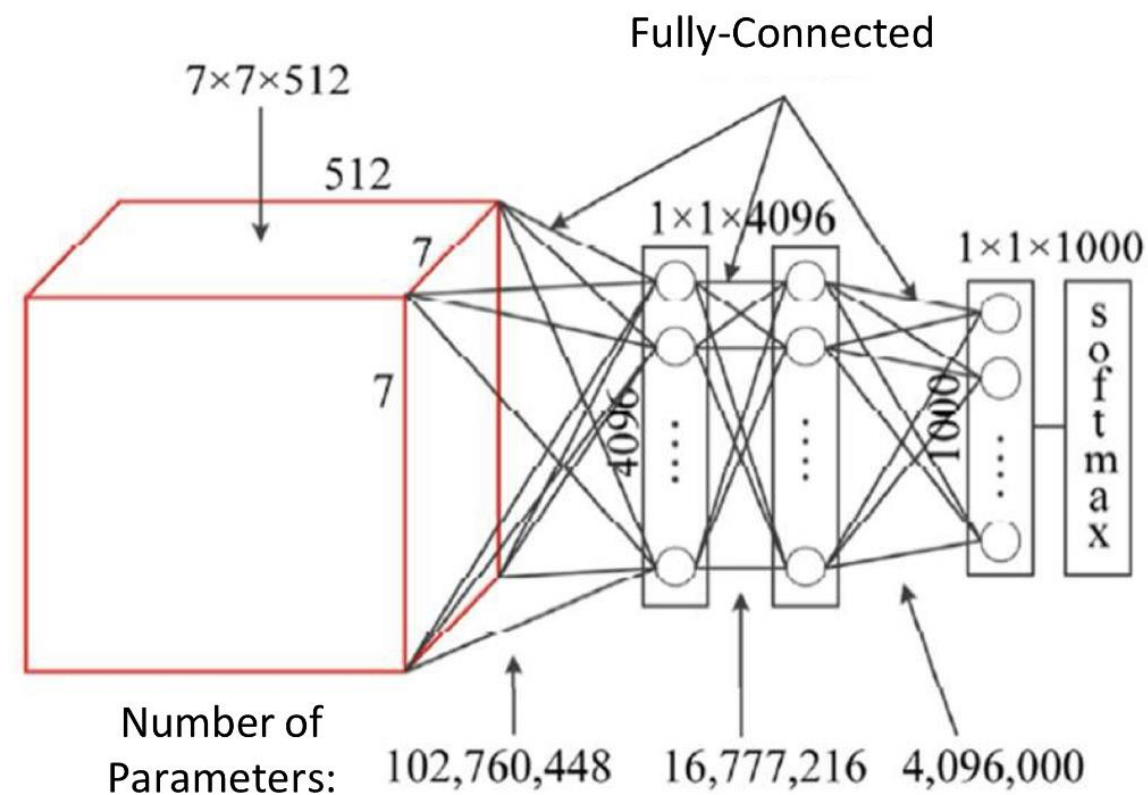
- More layers do not imply more parameters * 알아만 두기!

→ 참고)

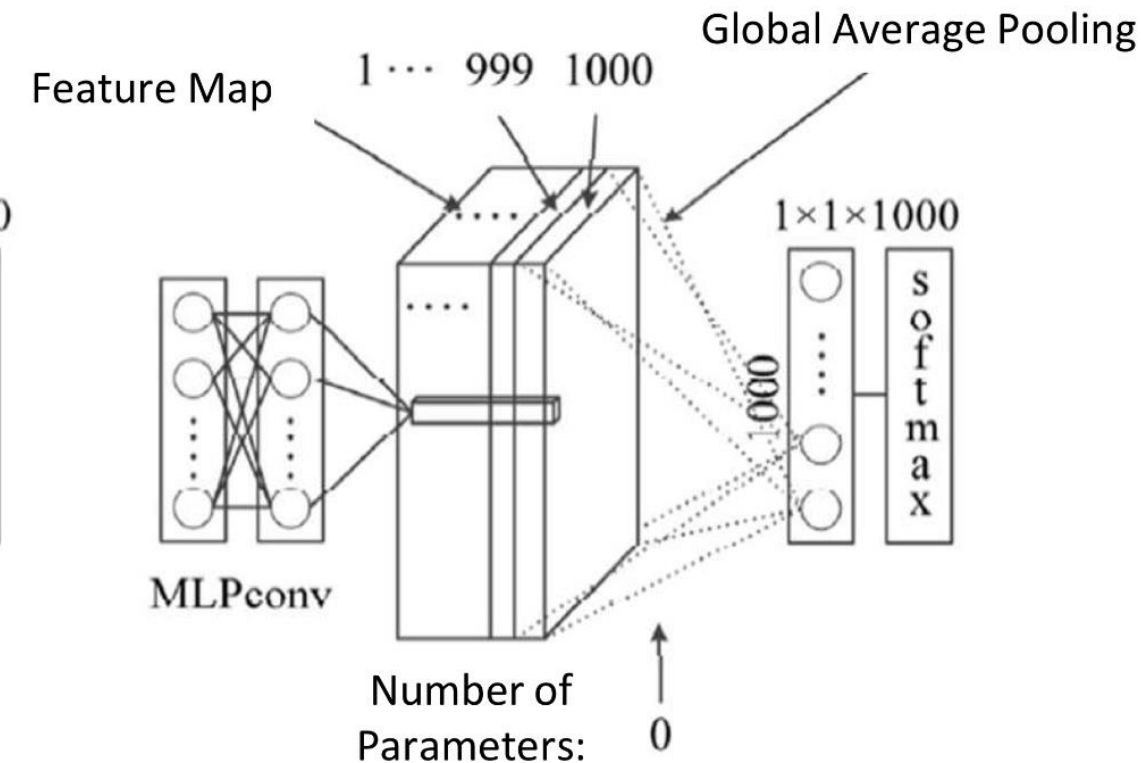
1. 1*1 conv block (차원[채널 개수] 변환)
2. Global Average Pooling

VGGNet
(19 layers,
138M)

GoogLeNet
(22 layers,
4M)



(a) Fully-Connected Layers in VGGNet

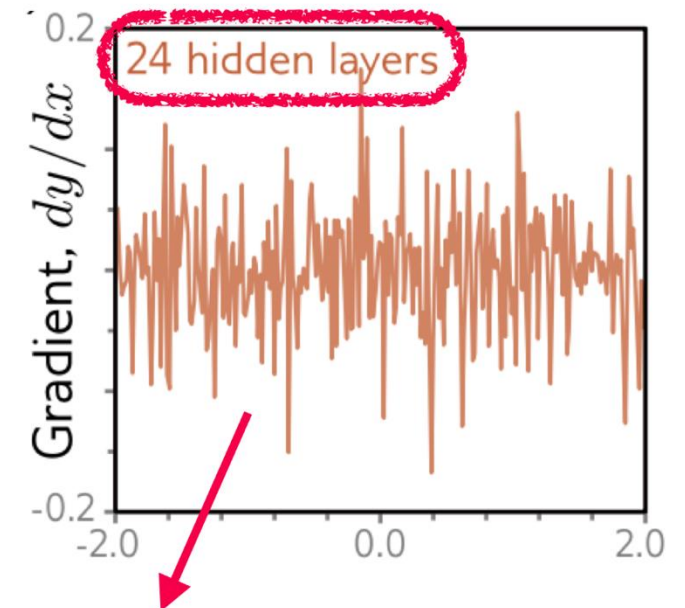
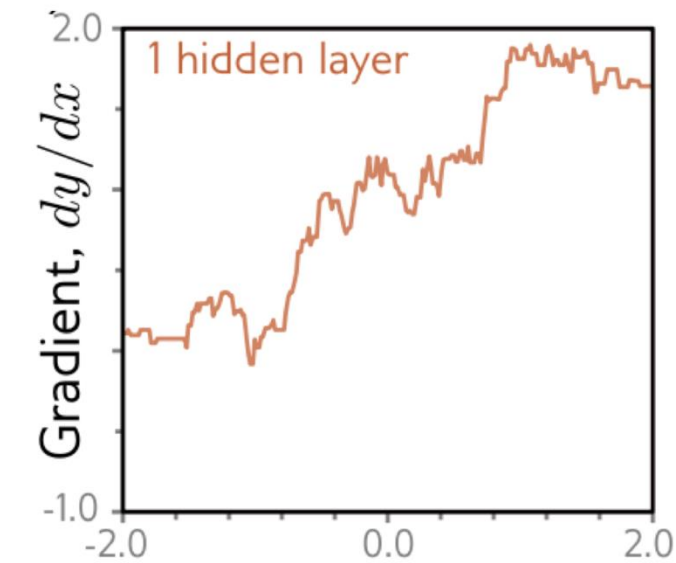
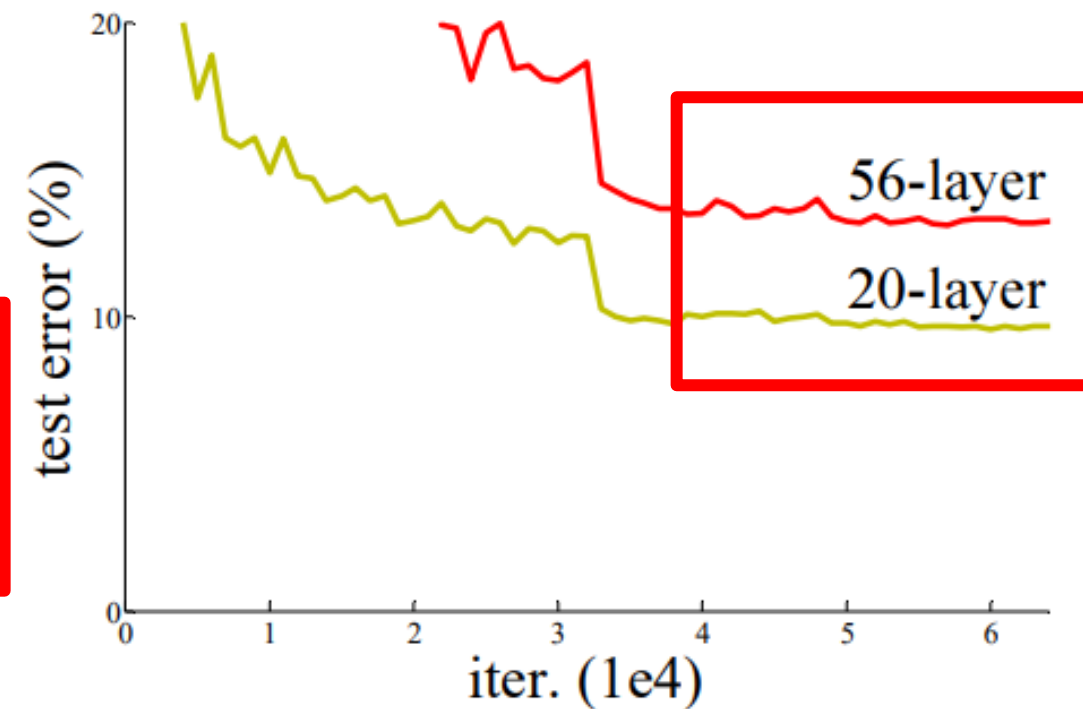
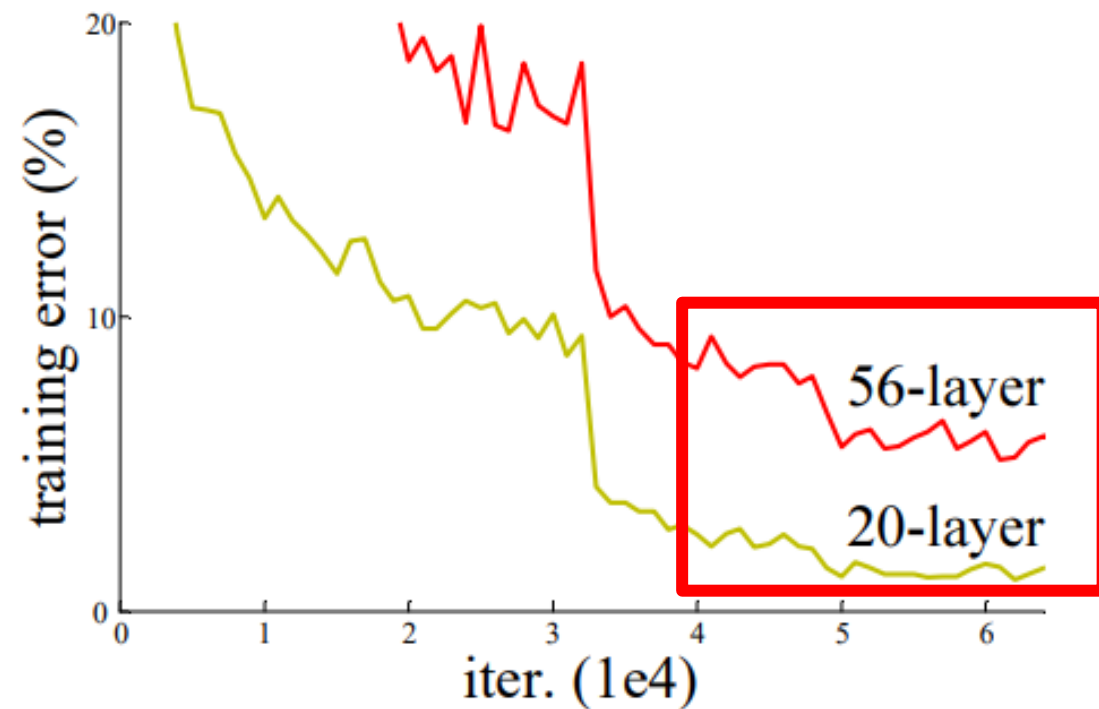


(b) Global Average Pooling in NIN

03. Deeper Layers

Deeper Layers

- Deeper layers are more difficult to **optimize** (i.e., NOT overfitting problem!)



Deeper layer를 가진 neural net은 train & test 모두에 대해 성능 악화

Deep Residual Learning for Image Recognition, CVPR (2016)

... gradient are shattered as we increase the number of layers

Deeper layer에서의 gradient는 매우 빠르게 변합니다.

03. Deeper Layers

Deep Residual Learning for Image Recognition, CVPR (2016)

ResNet: Residual Blocks

- Add identity map after nonlinear function: $f(x) \rightarrow x + f(x)$

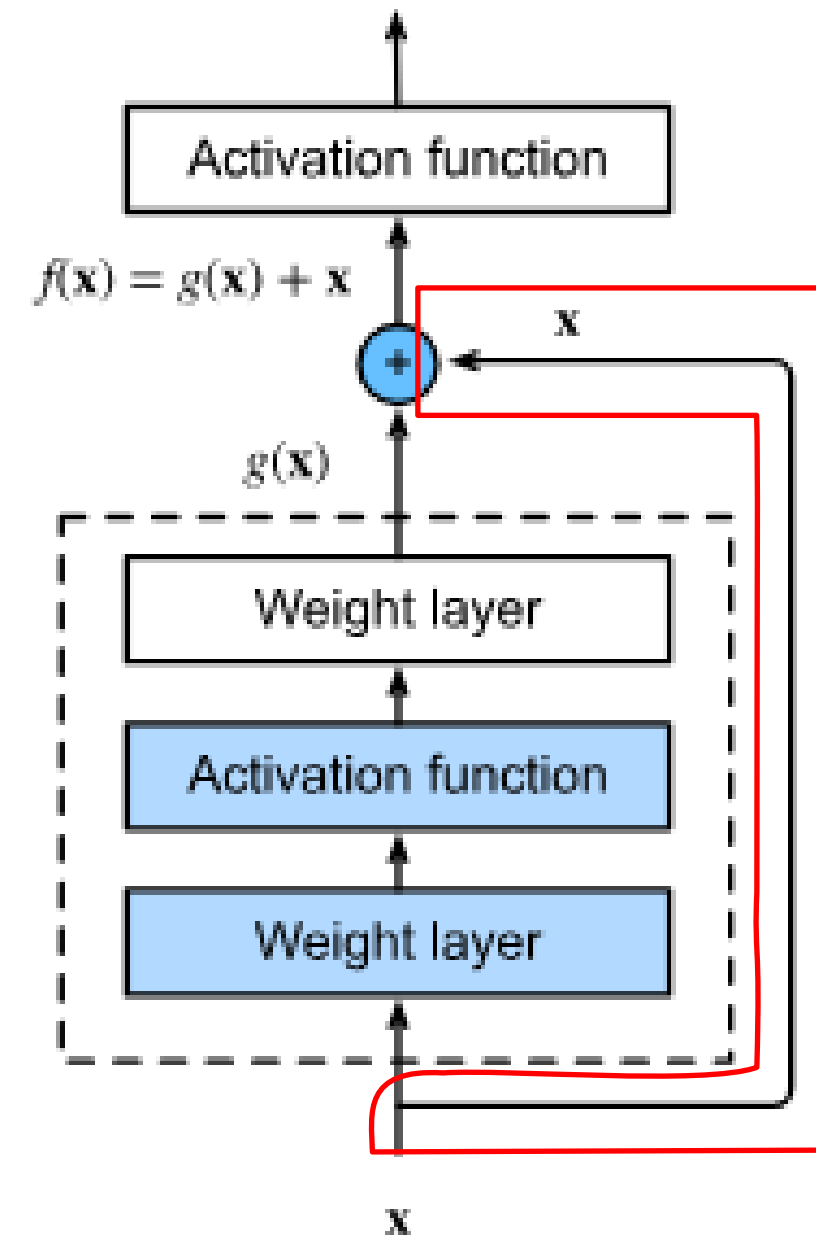
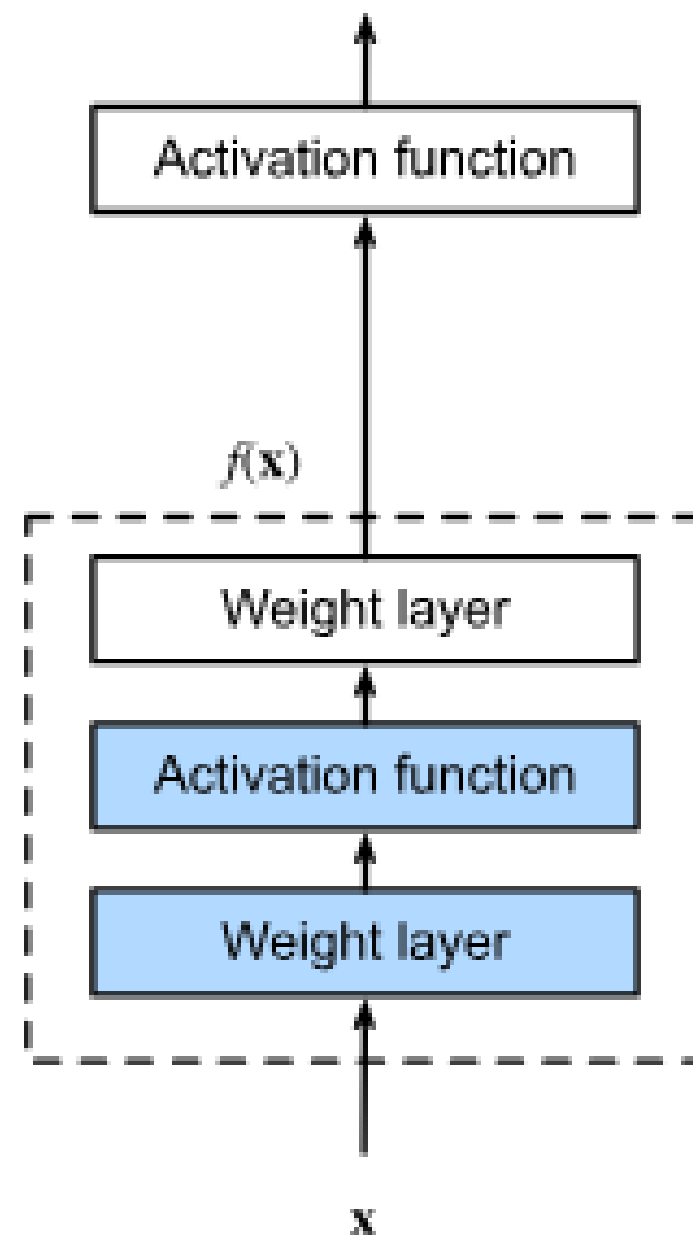
Plain Neural Nets

$$\mathbf{h}_1 = \text{Net}(\mathbf{x}; \theta_1)$$

$$\mathbf{h}_2 = \text{Net}(\mathbf{x}; \theta_2)$$

$$\mathbf{h}_3 = \text{Net}(\mathbf{x}; \theta_3)$$

⋮



Network with
residual connection

$$\mathbf{h}_1 = \mathbf{x} + \text{Net}(\mathbf{x}; \theta_1)$$

$$\mathbf{h}_2 = \mathbf{h}_1 + \text{Net}(\mathbf{x}; \theta_2)$$

$$\mathbf{h}_3 = \mathbf{h}_2 + \text{Net}(\mathbf{x}; \theta_3)$$

⋮

(Also called as
skip connection)

03. Deeper Layers

Deep Residual Learning for Image Recognition, CVPR (2016)

ResNet: Residual Blocks

- Add identity map after nonlinear function: $f(x) \rightarrow \mathbf{x} + f(x)$

Plain Neural Nets

$$\mathbf{h}_1 = \text{Net}(\mathbf{x}; \theta_1)$$

$$\mathbf{h}_2 = \text{Net}(\mathbf{x}; \theta_2)$$

$$\mathbf{h}_3 = \text{Net}(\mathbf{x}; \theta_3)$$

⋮

Network with **residual connection**

$$\mathbf{h}_1 = \mathbf{x} + \text{Net}(\mathbf{x}; \theta_1)$$

$$\mathbf{h}_2 = \mathbf{h}_1 + \text{Net}(\mathbf{x}; \theta_2)$$

$$\mathbf{h}_3 = \mathbf{h}_2 + \text{Net}(\mathbf{x}; \theta_3)$$

⋮

$$\mathbf{y} = \text{Net}_L(\text{Net}_{L-1}(\cdots (\text{Net}_2(\text{Net}_1(\mathbf{x}; \theta_1); \theta_2) \cdots); \theta_{L-1}); \theta_L)$$

$$\longrightarrow \frac{\partial \mathbf{y}}{\partial \mathbf{h}_1} = \frac{\partial \mathbf{y}}{\partial \mathbf{h}_L} \frac{\partial \mathbf{h}_L}{\partial \mathbf{h}_{L-1}} \cdots \frac{\partial \mathbf{h}_2}{\partial \mathbf{h}_1} \quad \text{product of gradients}$$

Deeper layers suffer less from shattered gradients

$$\mathbf{y} = \mathbf{x} + \text{Net}_1(\mathbf{x}; \theta_1) + \text{Net}_2(\mathbf{x} + \text{Net}_1(\mathbf{x}; \theta_1); \theta_2) + \cdots$$

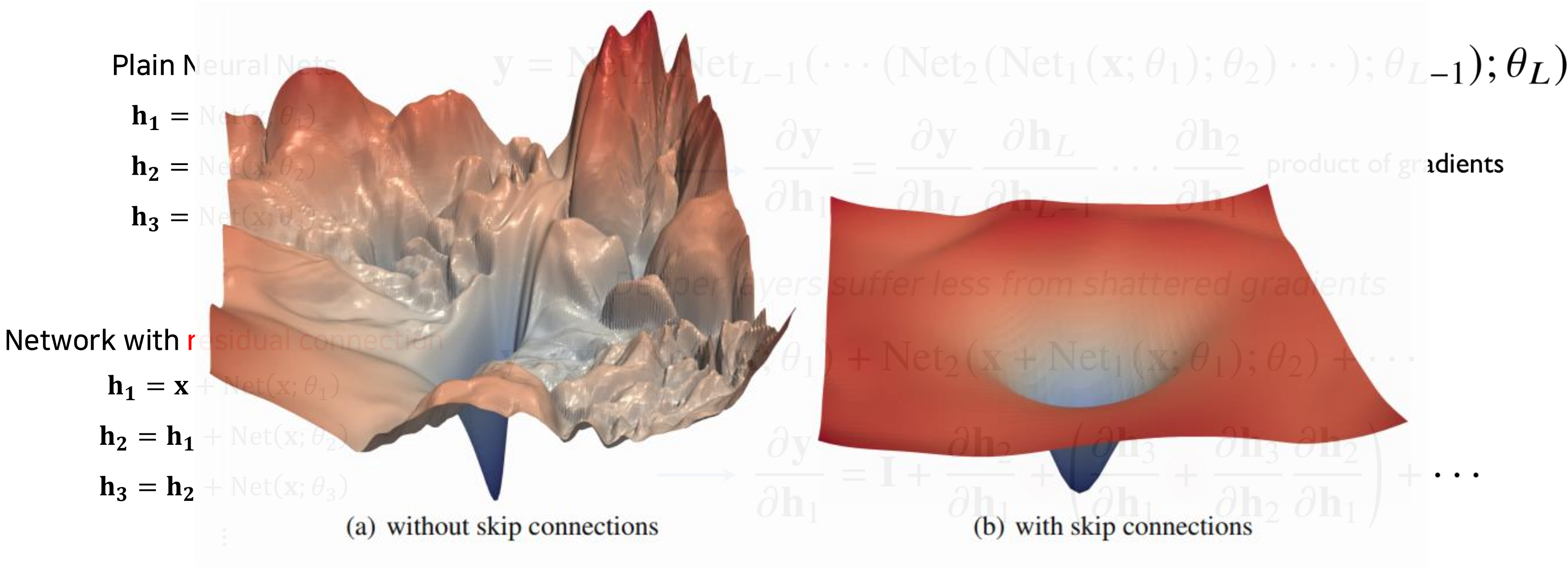
$$\longrightarrow \frac{\partial \mathbf{y}}{\partial \mathbf{h}_1} = \mathbf{I} + \frac{\partial \mathbf{h}_2}{\partial \mathbf{h}_1} + \left(\frac{\partial \mathbf{h}_3}{\partial \mathbf{h}_1} + \frac{\partial \mathbf{h}_3}{\partial \mathbf{h}_2} \frac{\partial \mathbf{h}_2}{\partial \mathbf{h}_1} \right) + \cdots$$

03. Deeper Layers

Deep Residual Learning for Image Recognition, CVPR (2016)

ResNet: Residual Blocks

- Add identity map after nonlinear function: $f(x) \rightarrow x + f(x)$

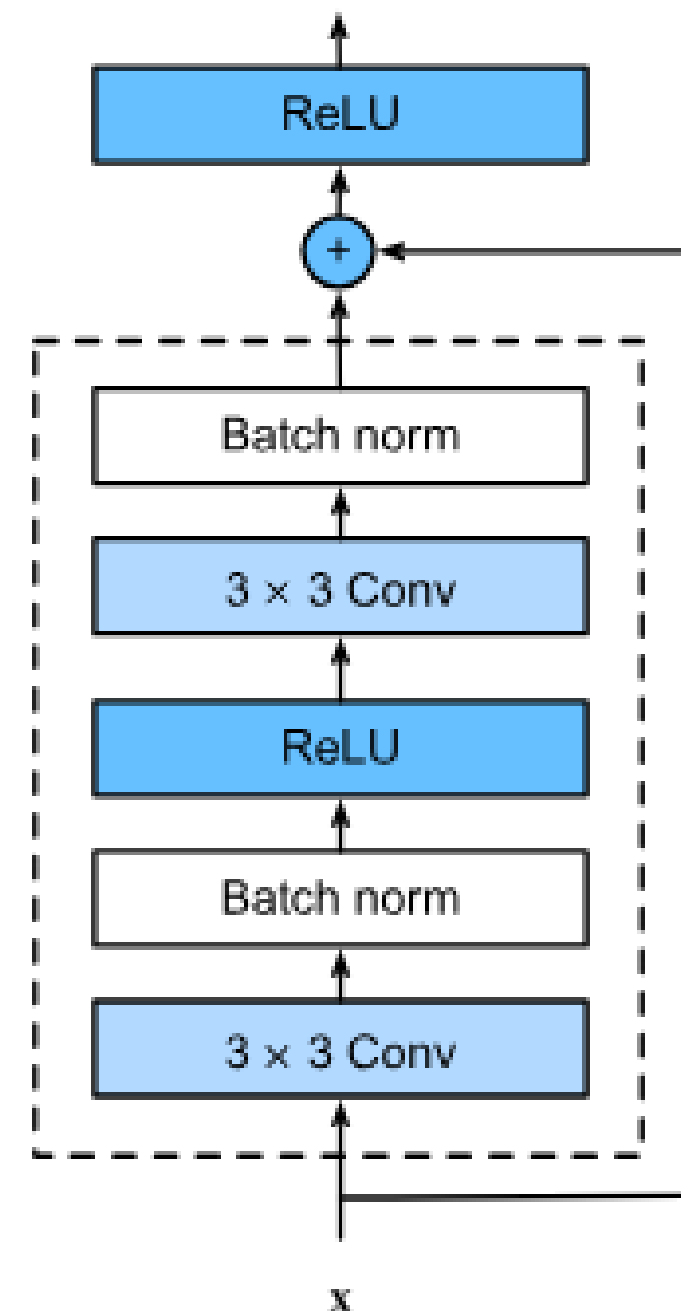
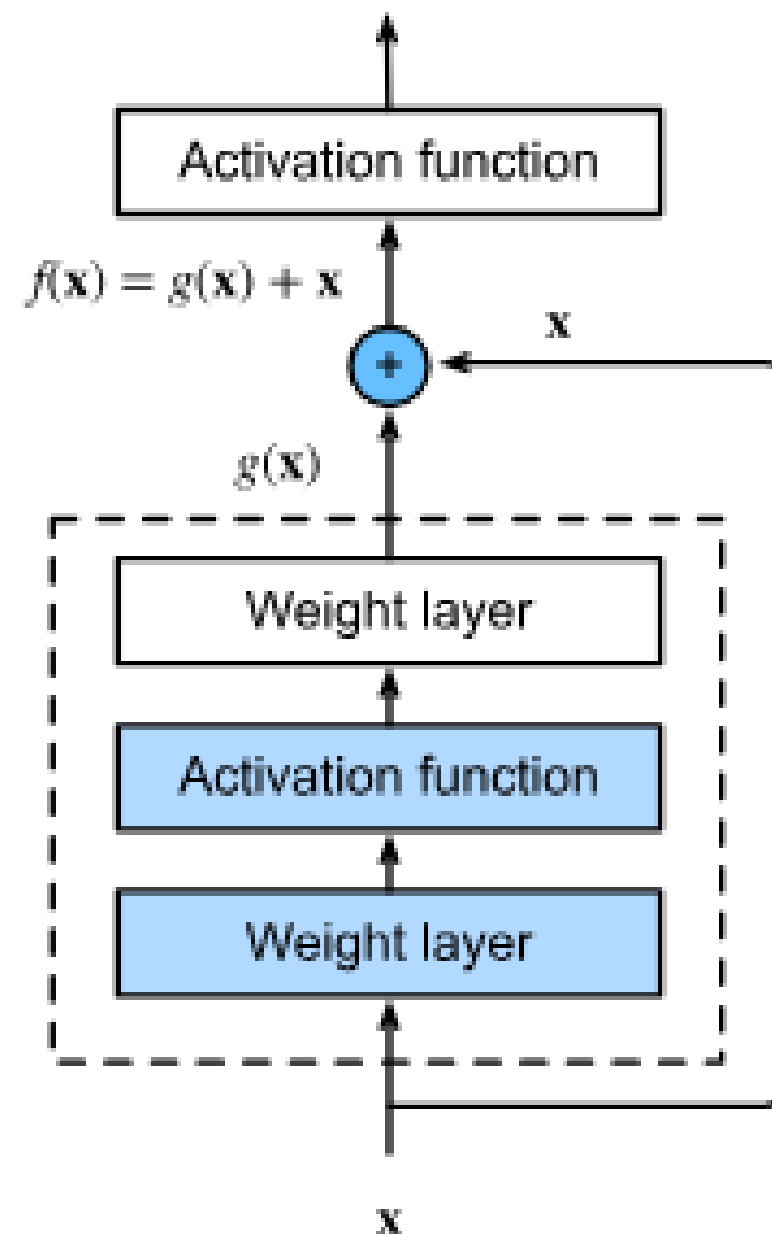


03. Deeper Layers

Deep Residual Learning for Image Recognition, CVPR (2016)

ResNet: Residual Blocks

- Residual block: Conv + ReLU + Conv + Skip



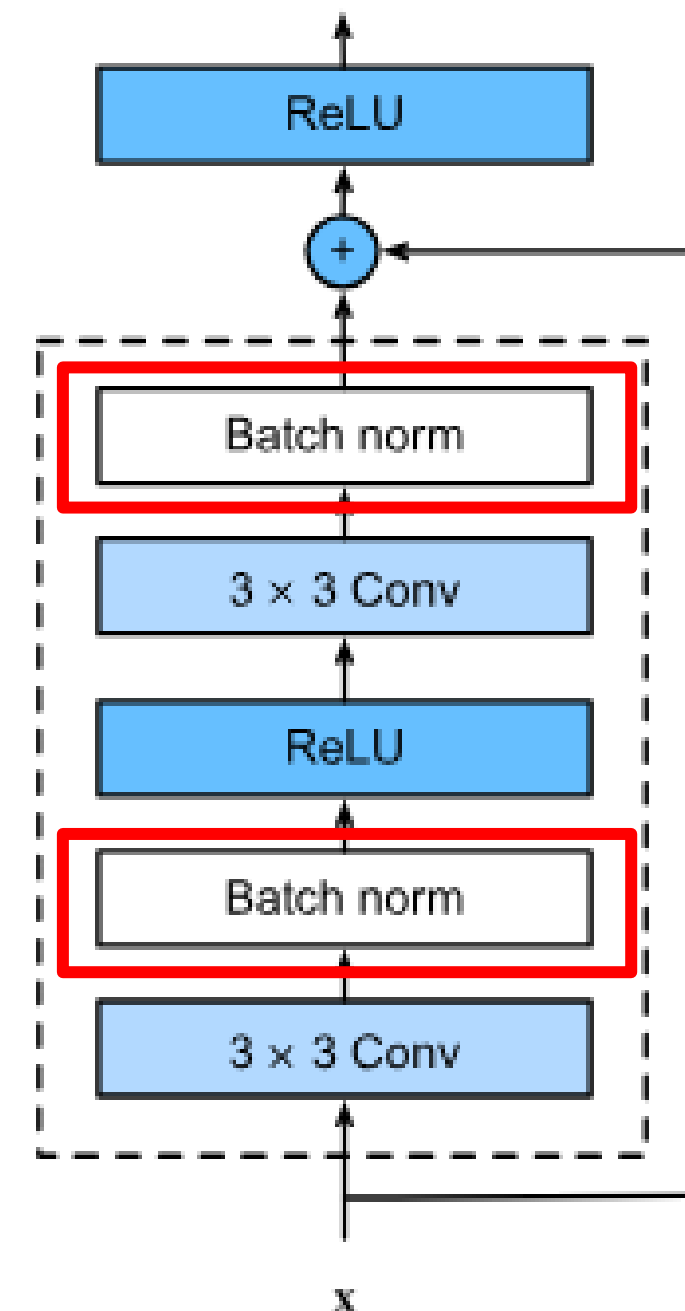
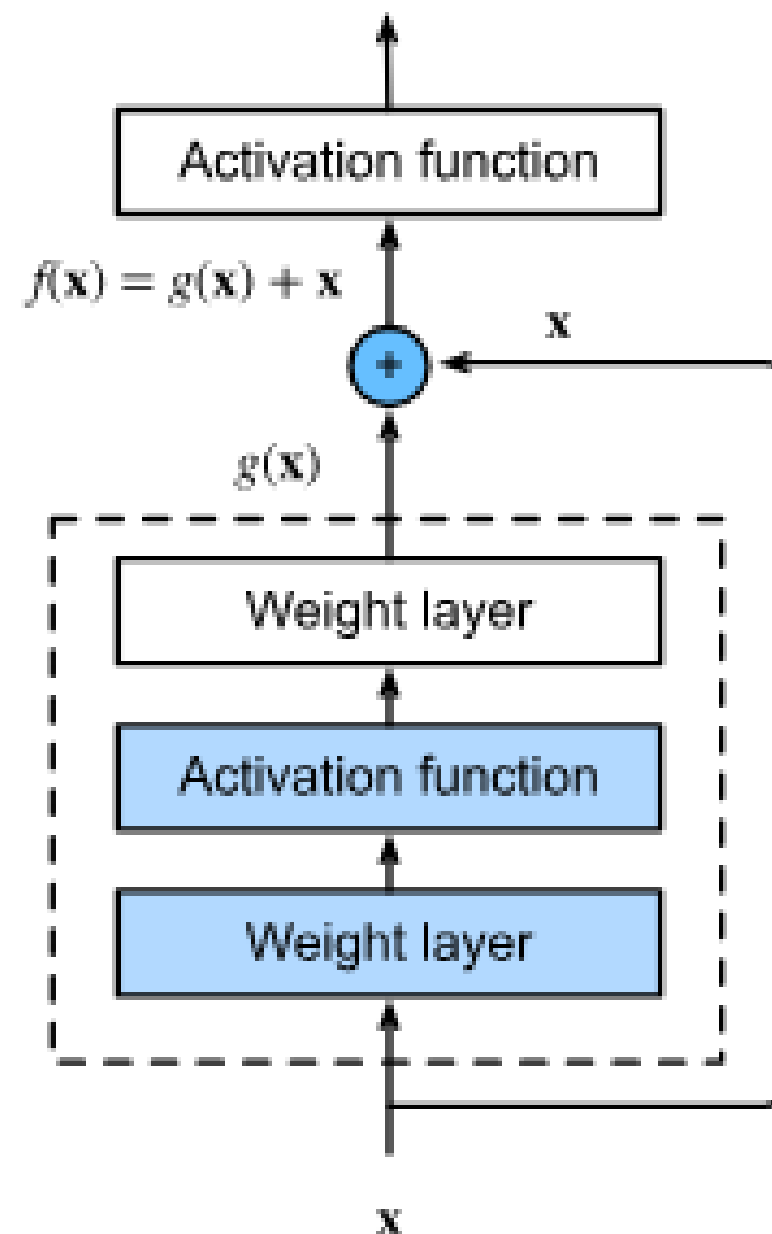
type I: direct shortcut
(* projection shortcut도 있는데
이건 1x1 Conv 배운 이후에 한번 보세요!)

03. Deeper Layers

Deep Residual Learning for Image Recognition, CVPR (2016)

ResNet: Residual Blocks + BatchNorm

- Residual block: Conv + BatchNorm + ReLU + Conv + BatchNorm + Skip



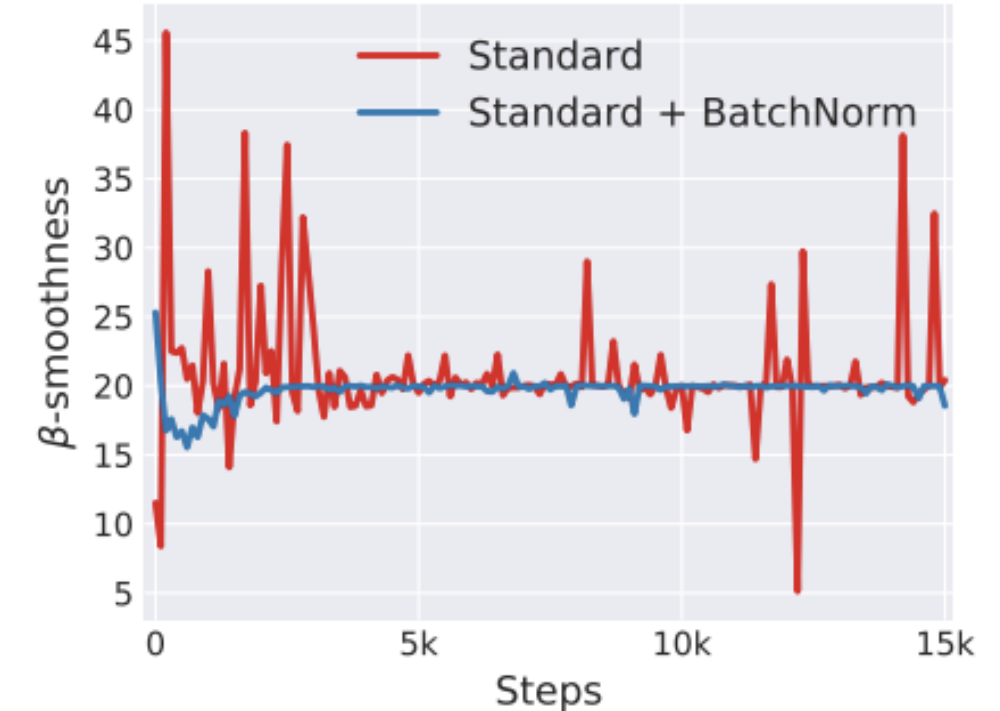
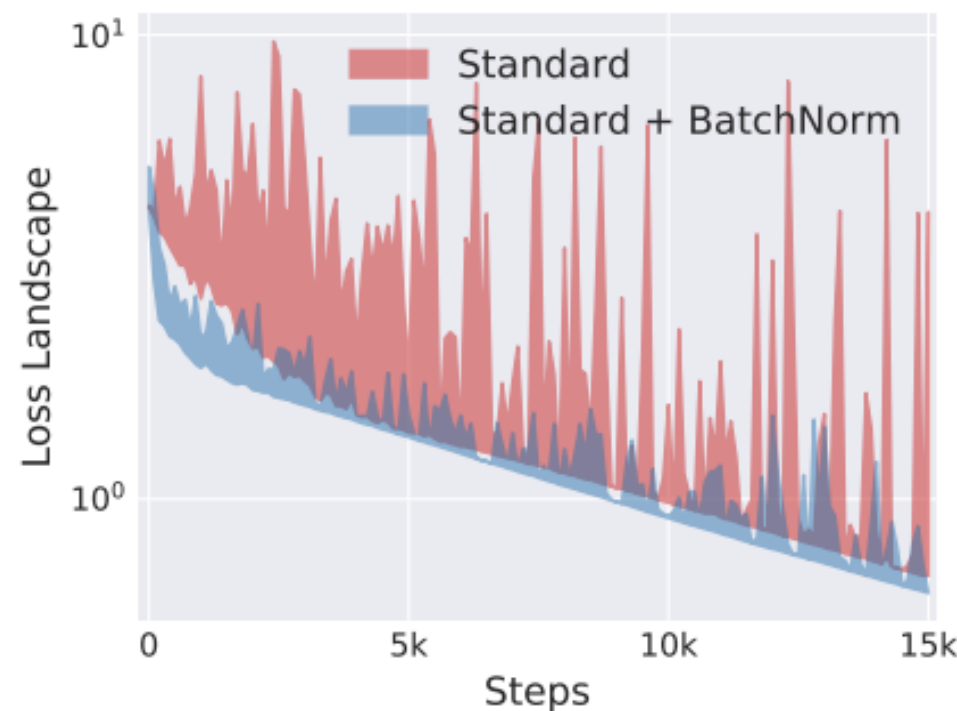
type I: direct shortcut
(* projection shortcut도 있는데
이건 1x1 Conv 배운 이후에 한번 보세요!)

03. Deeper Layers

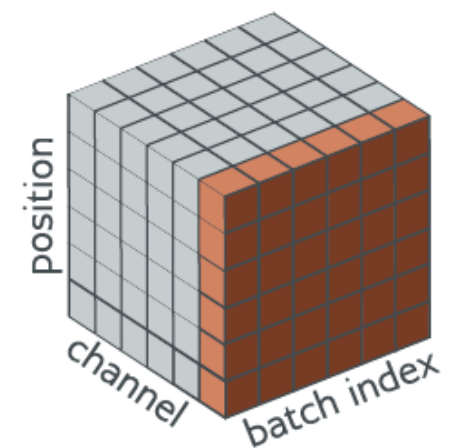
Deep Residual Learning for Image Recognition, CVPR (2016)

ResNet: Residual Blocks + BatchNorm

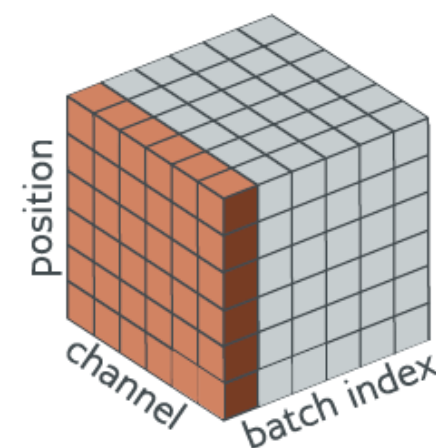
- BatchNorm helps optimization



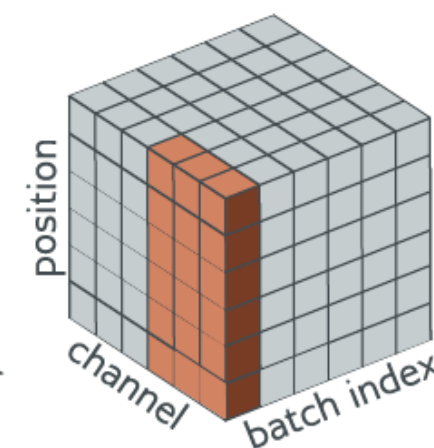
- Activations-based Layers
 - batch normalization (BN)
 - layer normalization (LN)
 - group normalization (GN)
 - instance normalization (IN)



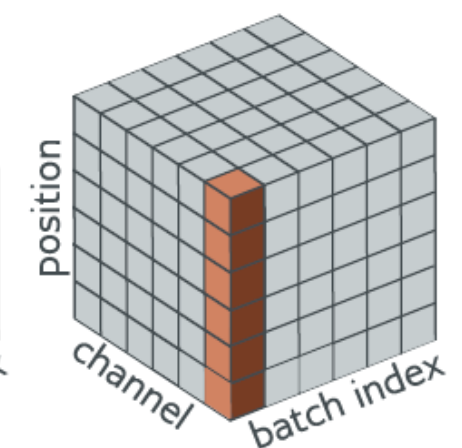
BatchNorm
Ioffe & Szegedy (2016)



LayerNorm
Ba et al. (2016)



GroupNorm
Wu & He (2018)



InstanceNorm
Ulyanov (2016)

04

Sequential Data

이어서 ..